

The Assertion Ladder: Graduated Assertion for Embedding a Technical Domain in Repositories Shared with Coding Agents

Leigh Griffin*, Paul Power*, Ray Carroll*, Colm Dunphy[†], Darrin Taylor[†], and Eamonn de Leastar[†]

*Red Hat, Waterford, Ireland

[†]South East Technological University (SETU), Waterford, Ireland

ABSTRACT Coding agents now contribute to repositories alongside people. They write quickly, they start every session with no memory of the last one, and their changes arrive faster than a person can review them closely. Two forces follow. *Sprawl* is a large volume of locally plausible changes spreading one domain rule across many places. *Cognitive erosion* is neither the reviewer nor the agent holding enough of the domain in mind to notice. Current practice answers with architecture checks, executable specifications and instruction files for agents, but it has no standard way to combine them. There is no shared vocabulary for how strongly a rule is asserted, no link from agent instructions to enforcement, no closure between requirements and scenarios, and no benchmark of whether a repository's own harness catches the mistakes that matter. We propose the *Assertion Ladder*, a pattern whose six rungs run from a rule that is merely implied, through one that is shown, told, checked and explained, to one that cannot be written wrongly at all, realised by nine smaller patterns. Read through Lean thinking, the ladder is standard work, with its tests built in, for a contributor who cannot remember it. We show it in two settings. A greenfield service, presented in TypeScript with equivalent Java and Python implementations, binds every rung; seeding fourteen canonical mistakes into each implementation, its single verify command catches all 42, where before the requirement and behavioural rungs it caught 30 of 36. A production TypeScript platform, Tutors, shows the ladder retrofitted while agents were already contributing, and its release checks extracted into a reusable harness. The repositories are public.

KEYWORDS Coding agents, Software architecture, Architecture testing, Behaviour-driven development, EARS, Lean software development, Design patterns, Human-AI collaboration.

1. Introduction

A new kind of contributor has joined software teams. Coding agents, language models given a shell, a file system and a test runner, now read a repository, plan a change, edit many files and run the tests before a person sees the result (M. Chen et al. 2021; Jimenez et al. 2024; Yang et al. 2024). They are fast

and broadly competent. They are also, in one respect, the most difficult colleague a team has ever had: every session starts with no memory of the last one. Whatever the agent learned about the domain yesterday, about why the work-in-progress limit is checked in the service rather than the route, or why the domain objects never import the web framework, is gone this morning.

The only thing a person and an agent reliably share is the repository. Whatever the repository does not say, and does not enforce, the agent will reinvent, usually plausibly and often differently each time. People have the opposite problem. They remember the domain, but they cannot read every line an agent writes. Studies of programmers working with code-generating tools describe a shift from writing to reviewing, and reviewing under that volume slides towards skimming (Vaithilingam et al.

JOT reference format:

Leigh Griffin, Paul Power, Ray Carroll, Colm Dunphy, Darrin Taylor, and Eamonn de Leastar. *The Assertion Ladder: Graduated Assertion for Embedding a Technical Domain in Repositories Shared with Coding Agents*. Journal of Object Technology. Vol. vv, No. nn, yyyy. Licensed under Attribution 4.0 International (CC BY 4.0)
<http://dx.doi.org/10.5381/jot.yyyy.vv.nn.aa>

2022; Barke et al. 2023). Security studies show what can pass unnoticed (Pearce et al. 2022; N. Perry et al. 2023).

We give the two resulting forces names. *Sprawl* is what happens when a large volume of locally plausible changes spreads one domain rule across many places. A check is copied into a route because it was nearby. A notification is called inline because that was quicker than publishing an event. A framework annotation lands on a domain object because it was convenient. Every one of those changes would pass review on its own. Taken together they leave the domain in pieces. *Cognitive erosion* is what happens when neither party can hold enough of the domain in mind to notice: the reviewer checks syntax because the intent is not visible in the diff, and the agent re-derives conventions from scratch every session. Lehman observed long ago that a system's structure decays under change unless work is done to preserve it (Lehman 1980). Agents have not repealed that law. They have made it run faster.

Software has met volume before. The case that Lean's answer to work done quickly by many hands applies to software, and that it is needed more now that so much code is produced so fast, is one the first author makes at book length (Griffin 2026b). This paper takes one piece of it, the repository, and works it through.

Where we meet the problem This paper comes out of a collaboration between a university and an industrial engineering site in the same city, and we meet the problem from both sides. At South East Technological University (SETU), students on the online Higher Diploma in Science in Computer Science¹ now learn to program alongside coding agents, and their lecturers need codebases that teach a domain to both. SETU's Master of Business Studies in Lean Enterprise Excellence² teaches the Lean practice this paper draws on. SETU delivers course material through Tutors, an open-source learning platform whose production TypeScript codebase coding agents were already changing (Tutors contributors 2026a). At Red Hat, the same agents contribute to engineering work, and hiring is moving towards graduates who can work safely in repositories that agents also change. The question is the same for a student, a new hire and an agent: how does a repository make its domain rules impossible to miss and hard to break, without burying everyone in prose? An agent with no limits will build past what a student can yet read, and past what a team can hold in mind. In both places the ladder protects what the people need to keep: the student's learning and the team's shared model of the system.

The need for constraints The common response is to write more guidance: longer prompts, longer contributor guides, longer instruction files for agents (AGENTS.md: A Simple, Open Format for Guiding Coding Agents n.d.). Guidance helps, but we have watched it fail in two ways. It is read at the start of a session and then competes with everything else in the agent's context, and it grows until nobody, person or agent, reads it at all. Our own first step beyond guidance was a repository in

which conventions are carried three ways at once: exemplified by one small vertical slice, declared in a manifest of a few hundred words, and enforced by architecture rules written as tests. The receivers in this paper began from that arrangement. This paper generalises it. It asks not only *whether* a rule is written down but *how strongly* it is asserted, and it extends the question from structure to behaviour and requirements.

The Assertion Ladder Our central claim is a teaching one. A coding agent is a capable learner with no memory, and the repository is its teacher. Teachers do not assert everything the same way. They show a worked example, they tell a rule, they set an exercise that marks the answer, they give feedback that explains the mistake, and now and then they arrange the task so that the wrong answer is simply not available. The *Assertion Ladder* (Figure 1) gives each of these moves a rung, from **L0 Implied** to **L5 Prevented**, and asks whoever designs a repository to place every domain rule on a rung on purpose: high enough that the learner cannot miss it, and no higher than the rule's risk justifies, because every assertion costs something to write, to maintain and to read.

Lean as the explanation None of this is new engineering. It is old engineering applied to a new worker. Lean thinking starts from *value*, what the customer and, here, the domain need, and treats everything that does not serve it as waste (Womack & Jones 1996; Poppendieck & Poppendieck 2006). For work that repeats, its central tool is *standard work*: the current best method, followed by everyone who does the job and improved by the team that owns it (Liker 2004). It reads a process as a *value stream* and asks where along it a defect is found and what the waiting and rework cost (Rother & Shook 1999). And it answers each problem with *root cause analysis*, asking why until the answer is something the process can change (Ohno 1988). Read this way, the rungs are standard work for a contributor who cannot remember it: shown in a worked slice, told in a short manifest, and then built into the work as checks that stop the build, explain the stop, or make the mistake impossible to write. Section 8 makes the case that Lean explains why the ladder works, rather than decorating it.

Critical gaps Software engineering already has most of the parts. Architecture conformance checking, specification by example, structured requirements and instruction files for agents all exist, and Section 9 reviews them. What it lacks is a standard way to combine them. We identify five gaps in current practice that call for standardisation. There is no shared vocabulary for how strongly a rule is asserted (G1). Instruction files for agents are unvalidated prose with no link to enforcement (G2). Conformance tools say nothing about which rules to check rather than document, or about a failure naming its fix (G3). Agent-facing repositories have no executable closure between requirements and the scenarios that test them (G4). And there is no benchmark of whether a repository's own harness catches the mistakes that matter, as opposed to whether an agent can complete a task (G5). None of these gaps is large on its own. Their combined effect is that every team sharing a repository with agents invents its own answer, and most answer with more

¹ <https://www.setu.ie/courses/higher-diploma-in-computer-science-2-years-online>

² <https://www.setu.ie/courses/master-of-business-in-Lean-enterprise-excellence>

1 prose.

2 **Two known uses** We show the pattern in two settings. The
3 first is greenfield: one application, *taskboard*, a small Kanban
4 service with a work-in-progress limit, built with the ladder
5 from the start. We walk through its TypeScript implementation
6 and show that Java and Python implementations with identical
7 behaviour bind every rung equivalently (Griffin 2026a). We call
8 these implementations *receivers* of the pattern rather than its
9 source, and work backwards from what each gains to the single
10 capability they share. The second is a retrofit: *Tutors*, an open-
11 source TypeScript learning platform in production, to which
12 the rungs were added after the fact while coding agents were
13 already contributing, and from which the release checks were
14 then extracted into a reusable, product-independent harness
15 (*Tutors contributors* 2026a,b). One shows the ladder built where
16 nothing existed. The other shows it fitted to a codebase with
17 years of history.

18 **Contributions** The paper makes five contributions, each
19 aimed at one or more of the gaps.

- 20 1. The Assertion Ladder, a pattern in the classic form (Alexander
21 et al. 1977; Gamma et al. 1994) with six rungs and nine
22 rung patterns, proposed as a shared vocabulary for how
23 strongly a repository asserts each domain rule (G1, G2,
24 G3; Section 3).
- 25 2. A unification of three kinds of intent, structure, behaviour
26 and requirements, as three loads on the same ladder, with a
27 closure check that ties requirements to scenarios and the ob-
28 servation that the pattern of an EARS requirement (Mavin
29 et al. 2009) predicts the layer that should implement it (G4;
30 Sections 3 and 4).
- 31 3. A greenfield receiver in TypeScript, with equivalent Java
32 and Python receivers, showing which rungs each language
33 can reach and what it uses where it cannot (Section 4).
- 34 4. A retrofit of a production codebase, *Tutors*, and the extrac-
35 tion of its release checks into a reusable harness, showing
36 how the ladder is adopted where it was not designed in
37 (Section 5).
- 38 5. Violation seeding as a benchmark of a repository rather
39 than of an agent, run before and after the behavioural and
40 requirement rungs were added, and a pilot with eighteen
41 coding agents (G5; Section 7).

42 2. Background

43 The Assertion Ladder draws on five bodies of work: the emer-
44 gence of coding agents, architectural structure and its testing,
45 behaviour and requirement specification, the psychology of
46 learning, and Lean thinking. This section introduces only what
47 the later sections use.

2.1. Coding agents and the memoryless contributor

Code-generating models moved from completing a line
(M. Chen et al. 2021) to resolving whole issues in real reposi-
tories (Jimenez et al. 2024). What made the second step possible
was less the model than its harness: an agent that can list files,
open them, edit them and run the tests behaves very differently
from one that emits a single completion (Yang et al. 2024). Tool
vendors now converge on a convention for telling such an agent
how a repository works: a Markdown file at the root, loaded at
the start of every session (*AGENTS.md: A Simple, Open Format
for Guiding Coding Agents* n.d.). That file is the agent’s whole
inheritance from previous sessions.

Three properties of this contributor matter for what follows.
It is *capable*: given a clear pattern it will reproduce it across
layers and languages. It is *memoryless*: nothing learned in one
session survives to the next unless the repository records it. And
it is *fast*: its output can exceed what a person can review line
by line, which moves the person’s role from author to reviewer
(Barke et al. 2023) and makes the quality of what passes review
depend on what the reviewer can see.

2.2. Structure and its enforcement

Layering is one of the oldest structuring ideas in software (Di-
jkstra 1968); its rationale is information hiding, so that a deci-
sion likely to change is confined to one module (Parnas 1972).
The layered architecture of enterprise applications, with a pre-
sentation layer, a service layer, a data-access layer and a do-
main model, is catalogued as a pattern (Buschmann et al. 1996;
Fowler 2002), and domain-driven design places the domain
model at the centre of attention (Evans 2003). Hexagonal and
Clean Architecture invert the dependencies so that the domain
depends on nothing and outer adapters depend on ports it owns
(Cockburn 2005; Martin 2017).

Structure described only in prose decays. Architecture fitness
functions make an architectural characteristic executable, so that
a build can fail when it is violated (Ford et al. 2017); ArchUnit
does this for Java dependencies by analysing bytecode in an
ordinary unit test (TNG Technology Consulting n.d.). Such
rules are the structural core of our third rung.

2.3. Behaviour and requirements

Behaviour-driven development writes tests as examples of be-
haviour in the vocabulary of the domain, conventionally in the
form *given* a context, *when* an event occurs, *then* an outcome
is expected (North 2006); specification by example treats such
examples as the living specification of the system (Adzic 2011).

The Easy Approach to Requirements Syntax (EARS) con-
strains natural-language requirements to a small set of tem-
plates, each distinguished by a keyword (Mavin et al. 2009):
ubiquitous requirements (“The system shall. . .”), *state-driven*
(“While. . .”), *event-driven* (“When. . .”), *optional feature*
(“Where. . .”), *unwanted behaviour* (“If. . ., then. . .”) and *com-
plex* combinations of these. EARS was designed to remove
ambiguity for human readers. Section 3 argues that its key-
words do more work in a repository shared with agents: they
classify a rule in a way that predicts where its code belongs.

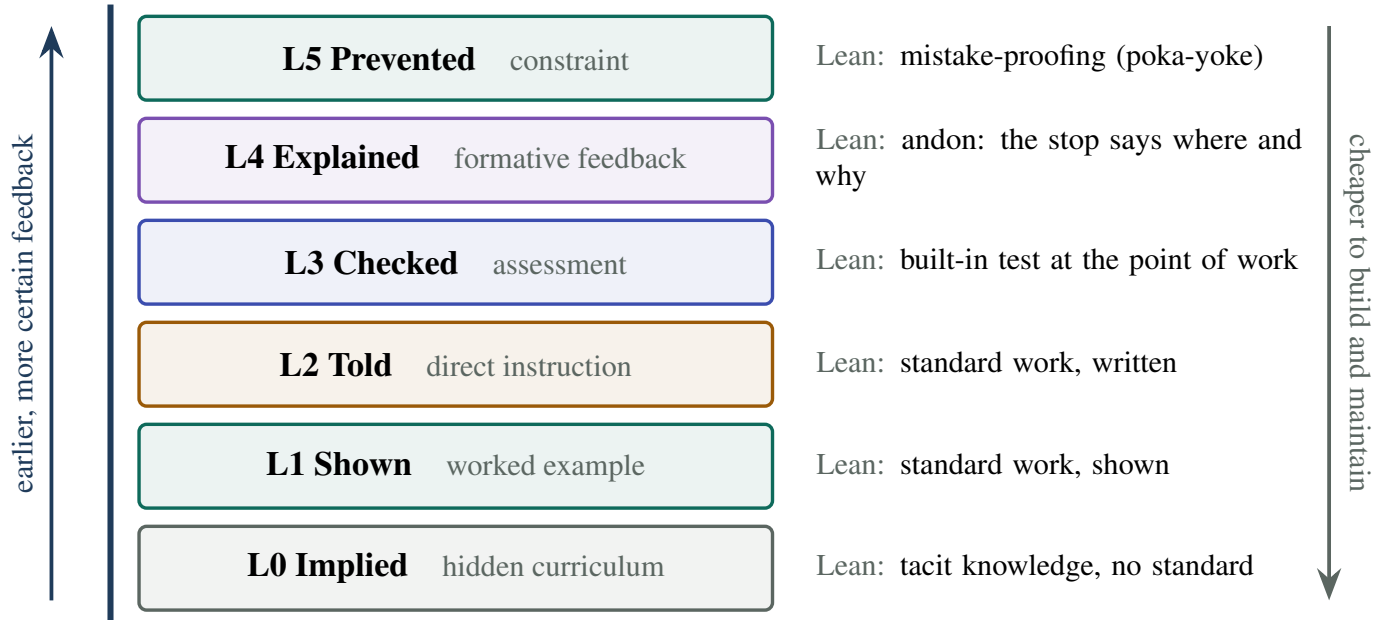


Figure 1 The Assertion Ladder. Each rung is a teaching move and a Lean mechanism. Rules climb only as far as their risk warrants; as a rule reaches a checked rung, it is removed from the prose below it.

2.4. Learning: worked examples, scaffolding and cognitive load

Cognitive load theory distinguishes the load a task imposes because of its intrinsic difficulty from load imposed by how the task is presented, and holds that working memory is small (Sweller 1988). One of its best-established findings is the worked-example effect: novices learn a procedure faster by studying a solved example than by solving the problem themselves (Sweller & Cooper 1985). Scaffolding describes support that lets a learner do what they could not do alone, and that is withdrawn as competence grows (Wood et al. 1976).

A coding agent is not a novice in the usual sense; it is highly competent in general and knows nothing of this repository in particular. The analogy we use is limited to that second property. The repository must teach the particular, every session, within a context window that is large but not unlimited.

There is a second learner in the room, and the agent changes what happens to them. An agent with no limits builds beyond the capability of the person it is helping. A student who asks for a feature gets one, across every layer, in code they cannot yet read, and in the next session the same student gets a different version of it. That is sprawl before there is a system to sprawl, and it takes away the practice through which mastery is built. This is not a problem of one course. We meet it as a culture and as an industry, in classrooms where students now program alongside agents and in teams where engineers do the same. What a student needs is not a slower agent but a repository that shows the one shape to copy, states the few rules that matter and marks the result, so that the feedback the agent gets is the feedback the student learns from. The same holds for a team. An engineering team cannot hold a system in mind that grows faster than its understanding of it, and the ladder protects the team's shared model the way it protects the student's learning. This

is the cross-reference that runs through the paper: a repository that teaches an agent the domain also teaches a person, and the discipline that keeps a classroom codebase learnable is the one that keeps a production codebase in one piece. It changes how we teach, and it changes how we work with people.

2.5. Lean thinking in software

Lean thinking began on the factory floor. The Toyota Production System, as described by its chief architect, set out to deliver what the customer wanted with as little waste as possible, and did so by building quality into each step of the work rather than inspecting it in at the end (Ohno 1988). Womack and Jones later restated it as five principles for any organisation: specify value from the customer's point of view, map the value stream, make the work flow, let the customer pull it, and keep improving (Womack & Jones 1996). Software took up the same ideas early (Poppendieck & Poppendieck 2003, 2006), and they now sit beneath continuous delivery and DevOps (Forsgren et al. 2018). This paper uses four of them, value, standard work, value stream mapping and root cause analysis, and three supporting mechanisms.

Value and waste Lean starts from value: what the customer actually wants. Anything that does not serve it is waste. The Poppendiecks translated the seven wastes of manufacturing into software as partially done work, extra features, relearning, handoffs, task switching, delays and defects (Poppendieck & Poppendieck 2006). An observational study of software teams found an overlapping but different set, including rework, unnecessarily complex solutions, extraneous cognitive load and knowledge loss (Sedano et al. 2017). For a repository shared with agents we add one clarification. The customer is not the only party with an interest in a change; the domain has one

1 too. A change that works for today's user but spreads a rule
2 across three places delivers value now and charges it back later
3 as rework. In this paper, *value* is working behaviour that keeps
4 the domain's rules in one place and in its own words, and *waste*
5 is anything that does not serve that end. Two wastes matter
6 most. Relearning, or knowledge loss, is the memoryless agent's
7 normal condition: each session begins by rediscovering what
8 the last one knew. Rework is what sprawl costs once it is finally
9 noticed.

10 **Standard work** Standard work is the current best known way
11 of doing a job, recorded so that everyone who does the job can
12 follow it and anyone who finds a better way can change it. It is
13 not a rulebook imposed from outside. In Toyota's own account
14 it is the foundation of improvement, because a job that nobody
15 does the same way twice cannot be improved, only varied (Liker
16 2004). Spear and Bowen, studying Toyota plants, found the idea
17 stated as a rule: all work is specified as to content, sequence,
18 timing and outcome, and the specification carries built-in tests
19 that signal a problem as soon as it occurs (Spear & Bowen 1999).
20 In software, the nearest equivalents are a team's agreed way of
21 adding a feature, the examples it copies, and the tests that check
22 the result.

23 **Value stream mapping** A value stream map draws every step
24 a piece of work passes through from request to delivery, and
25 marks which steps add value and which only wait, hand work
26 on or redo it (Rother & Shook 1999). The technique has been
27 adapted to software, where the material is information and much
28 of the waiting is invisible (Poppendieck & Poppendieck 2003;
29 Khurum et al. 2014; Ali et al. 2016). The view we need is short.
30 A change is written, verified, reviewed, merged and released.
31 The later a defect is found along that stream, the more steps it
32 has passed through and the more people have waited on it.

33 **Root cause analysis and improvement** Lean treats every prob-
34 lem as information about the process that produced it. Ohno's
35 practice was to ask *why* repeatedly, typically five times, until the
36 answer is a cause that a change to the process can remove rather
37 than a symptom to be patched (Ohno 1988). Toyota records
38 such an analysis on a single sheet, the A3 report, which follows
39 the plan-do-check-act cycle: state the problem, find its root
40 cause, choose a countermeasure, and check whether it worked
41 (Sobek II & Smalley 2008). The build-measure-learn loop of
42 the Lean startup applies the same cycle to product decisions
43 (Ries 2011). Small improvements made this way, one after
44 another, are what Lean calls *kaizen*.

45 **Supporting mechanisms** Three further terms appear briefly.
46 *Jidoka*, often translated as automation with a human touch,
47 means that a machine stops itself when something goes wrong,
48 so that nobody needs to watch it and no more bad work is made
49 until the cause is fixed (Ohno 1988). In software, it is the failing
50 build that blocks a merge. *Andon* is the signal that shows a stop
51 has happened and where; any worker may also pull the andon
52 cord to stop the line and ask for help. *Poka-yoke*, or mistake-
53 proofing, designs a step so that a mistake cannot be made, or is
54 caught at the moment it is made (Shingo 1986). In software, it
55 is a type that will not compile when a case is missing.

Lean in knowledge work, and its limits Lean transfers to 56
knowledge work, but not unchanged. At a large software ser- 57
vices firm, projects run on Lean principles outperformed com- 58
parable projects on most measures, yet uncertain tasks, invisible 59
work in progress and design problems that surface only during 60
implementation all made the manufacturing tools harder to ap- 61
ply (Staats et al. 2011). A review of practitioner reports found 62
Lean applied within agile teams in many ways and for different 63
purposes (Wang et al. 2012). Reinertsen argues more sharply 64
that product development is not repetitive manufacturing: some 65
of its variability creates value, and standardising that variability 66
destroys the value (Reinertsen 2009). We take the point. The 67
ladder standardises only the part of software work that ought 68
to repeat, such as where a rule lives, how an error is translated 69
and how a scenario cites a requirement, and leaves design to the 70
people and agents doing it. 71

2.6. Patterns 72

A pattern names a recurring problem in a context, the forces that 73
make it hard, and a solution that balances them (Alexander et al. 74
1977). The form was brought to software design by the Gang 75
of Four (Gamma et al. 1994) and to architecture by the POSA 76
series (Buschmann et al. 1996). We use it for the Assertion 77
Ladder because the ladder is not an algorithm or a tool but a 78
way of resolving forces that every team sharing a repository 79
with agents will meet. 80

3. The Assertion Ladder 81

This section states the pattern, describes each rung, names the 82
smaller patterns that realise the rungs, and shows that structure, 83
behaviour and requirements are three loads carried by the same 84
ladder rather than three separate techniques. 85

3.1. The pattern 86

Name	Assertion Ladder (also: Graduated Assertion).	87
Intent	Keep a technical domain intact when coding agents contribute, by asserting each domain rule at the weakest level that still covers its risk.	88 89 90
Context	A repository shared by people and coding agents that start every session with no memory. Changes arrive faster than anyone can review them closely.	91 92 93 94
Problem	How do a domain's rules survive contributors who cannot remember them and reviewers who cannot read everything?	95 96 97
Forces	See Figure 2. – A rule nobody wrote down is lost at the next session. – Written guidance grows until nobody reads it, and competes with everything else in an agent's context. – Every executable check costs effort to write, time in the build, and maintenance when the domain changes.	98 99 100 101 102 103 104 105 106

- The strongest assertions, those made by the type system, depend on the language.
- Too much constraint makes the repository hard to change. Ease of use is a force, not an afterthought.
- A check that an agent can edit is a check that an agent can weaken.

Solution For each domain rule, choose a rung. Climb until the rule’s risk is covered, then stop. When a rule reaches a checked rung, remove it from the prose so that the declaration stays short. Run every check behind one command, and put the checks themselves behind a human boundary.

Consequences

Feedback arrives early and names the fix; the manifest stays short; intent can be reviewed separately from code. In exchange there are more tests to maintain, the top rung differs between languages, and a rule that climbs further than its risk warrants makes change expensive.

3.2. The rungs

L0 Implied The rule exists only in someone’s head or in habit. The agent experiences nothing, and guesses. The guess is usually plausible, which is what makes L0 dangerous: nothing about the result looks wrong. Lean would call this tacit knowledge with no standard: nothing to follow, nothing to improve, and the cost of relearning paid every session. A teacher would call it the hidden curriculum.

L1 Shown One complete example embodies the rule: a vertical slice from the domain to the endpoint, small enough to read whole. Agents, like novices, copy structure before they understand it (Sweller & Cooper 1985), so the slice is the cheapest effective assertion there is. To a Lean eye the slice is standard work shown rather than written: the team’s current best way to add a feature, in a form a contributor can copy. Its weakness is that the learner may copy the wrong detail, or generalise from an accident of the example.

L2 Told A short declaration states the rule, in a file the agent’s tooling loads at the start of every session (*AGENTS.md: A Simple, Open Format for Guiding Coding Agents* n.d.). Prose can say what code cannot: what needs a person’s decision, which of two acceptable choices the team prefers, why a rule exists. It is the written half of standard work. Its weakness is that it can be forgotten, misread or crowded out, and that it grows.

L3 Checked An executable rule fails the build when the rule is broken. This is the rung of architecture tests, contract tests and behavioural scenarios. The agent experiences a red suite and must work out why. This is the built-in test that Spear and Bowen found attached to Toyota’s standard work (Spear & Bowen 1999): the defect is found where it is made, by the contributor who made it, before the change moves on to a reviewer, rather than by a separate inspection further down the stream.

L4 Explained The check’s failure names the rule, the offender and the fix. A bare assertion failure tells the agent *that* it is wrong; an explained failure tells it *what to do instead*, which is the difference between summative and formative feedback. A bare failure is a stop. An explained failure is a stop that, like an andon signal, shows where the problem is and why, and names the countermeasure. It costs a sentence per rule.

L5 Prevented The wrong form does not compile. A sealed hierarchy with an exhaustive switch, or a map keyed by a closed union type, makes an unmapped error impossible to write. This is mistake-proofing (poka-yoke) of the preventive kind: the fixture admits the part only one way, so the mistake is never made, where a check at L3 catches it once it has been. It is also the rung that depends most on the language.

3.3. Nine rung patterns

Each rung is realised by one or more smaller patterns, listed in Table 1. Two are not tied to a rung: SINGLE GATE, which runs every check behind one command so that “finished” has one meaning, and HUMAN BOUNDARY, which lists the few changes an agent may not make alone, including changes to the checks themselves.

CLOSURE CHECK recurs more than any other pattern, so it is worth a paragraph of its own. An *every X has a Y* assertion keeps two lists in step without either list knowing about the other: every domain error has an HTTP status; every repository implementation passes the same contract; every requirement has a scenario and every scenario cites a requirement. It is the cheapest way we know to make a growing set safe to extend, and extending sets is most of what an agent does all day.

3.4. Three kinds of intent, one ladder

Architecture tests, behaviour-driven development and structured requirements are usually presented as separate practices with separate literatures. In a repository shared with agents they are the same move applied to three kinds of intent (Table 2). *Structure* says where code may live and what it may touch. *Behaviour* says what the system does, in the domain’s own words. *Requirements* say why, once, in a form that can be traced. Each is declared, exemplified and enforced, and each has its own closure check.

3.5. EARS as a placement oracle

The most useful property of EARS in this setting was not one it was designed for. Its keywords classify a rule, and the class predicts where the rule’s code belongs (Table 3). An *unwanted behaviour* about one entity (“If a requested move is not allowed from the task’s status, then...”) is a guard on that entity that raises a domain error. A *state-driven* rule that spans many entities (“While the work-in-progress limit is reached...”) needs a collaborator and belongs in a service. An *event-driven* rule whose response is to tell someone (“When a task moves to done, the board shall announce...”) is a domain event published by the service with the effect in a handler. An *optional feature* (“Where a limit is configured...”) is a field on the one settings object. A *ubiquitous* rule (“The board shall answer every

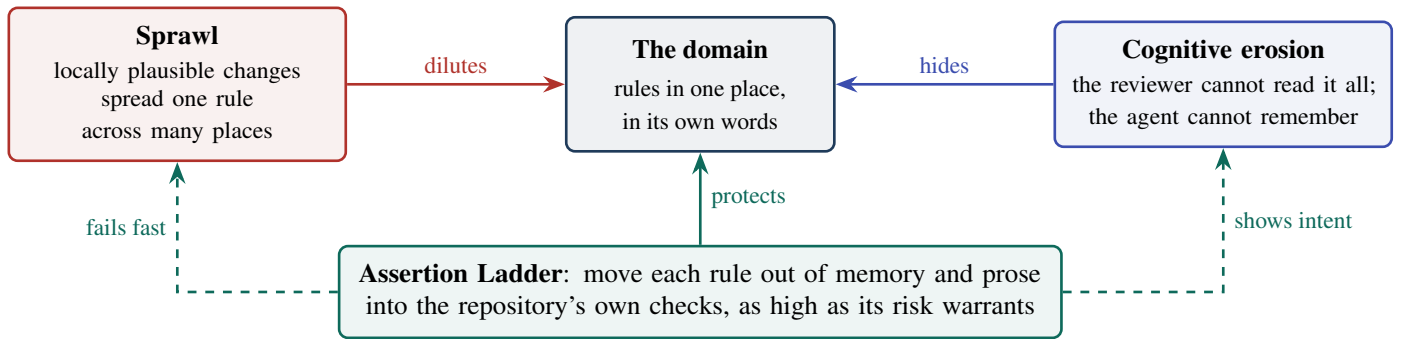


Figure 2 Two forces act on a domain once agents contribute. Sprawl spreads a rule across places; cognitive erosion hides the spread from both reviewer and agent. The ladder answers both by moving rules out of memory and prose into the repository’s own checks.

Table 1 The rung patterns. Each answers one recurring problem.

Rung	Pattern	Problem	Solution
L1	WORKED SLICE	How does a newcomer learn the shape of a feature?	One complete vertical slice, small enough to read whole.
L2	SHORT MANIFEST	How is what cannot be tested passed on?	One file of a few hundred words, loaded every session.
L3	EXECUTABLE RULE	How is a structural rule kept when nobody remembers it?	A test that fails when the rule is broken.
L3	CLOSURE CHECK	How do two lists stay in step?	Assert that every X has a Y.
L3	HONEST DOUBLE	How do fast tests stay truthful?	One contract, run against the real implementation and its stand-in.
L4	TEACHING FAILURE	How does a red build lead to the right fix?	The failure names the rule, the offender and the remedy.
L5	UNREPRESENTABLE MISTAKE	How is a mistake stopped before any test runs?	Types that make the wrong form fail to compile.
all	SINGLE GATE	When is work finished?	One command runs every rung; green means done.
all	HUMAN BOUNDARY	Who may change the ladder itself?	A short list of changes that need a person first.

refused request with a problem document”) is a cross-cutting invariant: one place, guarded by a check.

The mapping matters because the commonest placement mistake an agent makes is to put a correct rule in the wrong place, and a behavioural test cannot see that mistake (Section 7). A requirement written in EARS carries its placement with it. We present the mapping as a hypothesis supported by the ten taskboard rules and by one pilot run, not as a law.

3.6. Compression and fading

The solution says to climb only as far as a rule’s risk warrants, and to remove a rule from the prose once a check covers it. Both clauses are about ease of use, which is the force teams forget first. A manifest that repeats what the tests enforce is waste in the Lean sense: it costs reading time every session and adds nothing the gate does not already guarantee. Standard work is meant to change as the team learns, and here it changes by subtraction as well as addition: once a rule is built into a check,

the written standard no longer needs to carry it. Instructional design calls the same move *fading*: scaffolding is withdrawn as the learner no longer needs it (Wood et al. 1976). In the receivers the manifest says so of itself: “anything a test can check lives in the tests, not here.” Section 7 reports what the added rungs cost in words.

3.7. The agentic loop: trust, delegate, conform

The ladder is not only a way to grade rules. It is what lets a team hand work to an agent at all. The loop we run, in the receivers and in Tutors, has three moves. People *trust* the agent with a change because the rungs say what it cannot get wrong without the build saying so. They *delegate* the whole change rather than a sketch of it, and let the agent run the gate as often as it likes. And the gate *enforces conformity* to the ladder: a change is done when the single command is green, and a rule the agent cannot satisfy is raised to a person at the HUMAN BOUNDARY rather than relaxed. The loop closes when a miss

Table 2 The same three carriers hold three kinds of intent. Text colour follows the intent throughout the paper.

Intent	Declared (L2)	Exemplified (L1)	Enforced (L3–L5)
Structure : where code may live	a placement table in the manifest	the move-task slice, domain to route	architecture rules; exhaustive error mapping
Behaviour : what the system does	one convention line: scenarios cite ids and read given / when / then	the work-in-progress scenario	the scenarios, and the repository contract
Requirements : why	REQUIREMENTS.md, one EARS sentence per rule	ten taskboard rules with ids	the requirement closure check

Table 3 From EARS pattern to layer, with the taskboard rule that exercises each row.

EARS pattern	Implement as	Taskboard rule
Unwanted, one entity	a guard on the entity raising a domain error	R-FLOW-2: an invalid move is refused
State-driven, many entities	a service method with a collaborator	R-WIP-1: the work-in-progress limit
Event-driven, tells someone	a domain event; the effect in a handler	R-DONE-1: completion is announced
Optional feature	a field on the settings object	the configurable limit
Ubiquitous	one place, guarded by a closure or architecture check	R-ERR-1: every refusal is a problem document

shows up. A mistake that reached review is a rule sitting on too low a rung; the rung is raised, the check is added, and the next delegation starts from a stronger ladder. Section 7 shows one turn of the loop on the receivers. Section 5 shows it run for three months on a production codebase, where 329 of 483 commits were authored or co-authored by an agent and the gate that judged them grew, over the same months, from nothing to one required job.

4. A greenfield receiver in TypeScript

A pattern is only as credible as its known uses. This section presents the first: one application, *taskboard*, built with the ladder from the start. It exists three times with identical behaviour, in TypeScript, Java and Python. We call each a *receiver* because it does not define the Assertion Ladder; it takes the ladder on in the idiom of its language and gains something from it. We walk through the TypeScript receiver in detail, since it shares a language with the production retrofit in Section 5, and then show that the Java and Python receivers are equivalent: every rung pattern is bound in each, sometimes on a different rung. The second known use, a retrofit of a production codebase, follows in Section 5.

4.1. The application

Taskboard is a Kanban API with one entity. A task moves from *todo* to *in progress* to *done*, and may move back from *in progress* to *todo*. No more than a configured number of tasks may be in progress at once. When a task is completed the team is told. Every refused request is answered with an RFC 9457 problem document carrying a stable error code. The application is deliberately small: 283 lines of application code in Java, 223 in Python and 242 in TypeScript, down from 637, 729 and 957 in a first iteration that used dependency-injection containers,

object-relational mappers and a second entity. Small enough to read whole is a property the ladder depends on: the WORKED SLICE only works if the slice can be read in one sitting.

Each receiver is organised in the same six layers: *api*, *services*, *repositories*, *domain*, and the side layers *events* and *config*, with one composition root. Imports point only downwards, the domain imports nothing but the language's standard library, and each outside capability (HTTP, SQL, the environment, logging) belongs to exactly one layer.

4.2. A steel thread through all three kinds of intent

Figure 3 follows one rule, the work-in-progress limit, from its requirement to the gate. The requirement is written once in EARS:

R-WIP-1 (state-driven). While WIP_LIMIT tasks are in progress, when another task is moved to *in_progress*, the board shall refuse with *wip_limit_exceeded* and leave the task where it was.

Its pattern predicts a service method, because the rule needs to count other tasks. The scenario that covers it cites its id and reads as given / when / then (Listing 1). The domain error it raises carries a code and no HTTP; one handler maps it to a status, and a closure check refuses a new error without one. The limit arrives through a constructor from the one settings object. Every step is checked by the same command.

```
1 test("R-WIP-1: given a full board, when another
  task is
2   started, then it is refused and stays in
  todo", async () => {
3   const first = await service.createTask("First")
  ;
4   const second = await service.createTask("Second
  ");
```

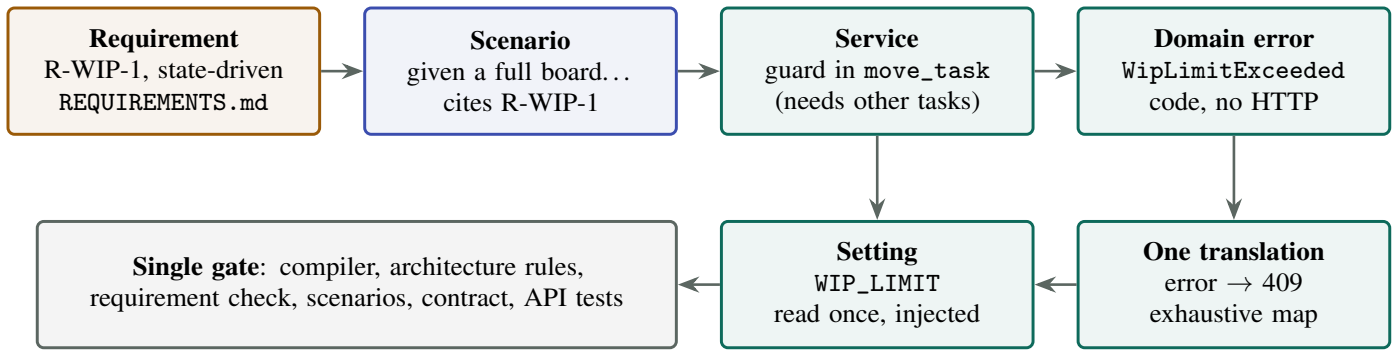


Figure 3 The work-in-progress limit as a steel thread. Colours mark the kind of intent each step carries; every step is checked by the single gate.

```

15  await service.moveTask(first.id, "in_progress")
16  ;
17  await expect(service.moveTask(second.id, "
18  in_progress"))
19  .rejects.toThrow(WipLimitExceeded);
20  expect((await service.getTask(second.id)).
21  status).toBe("todo");
22  });

```

Listing 1 A scenario in the TypeScript receiver. The title cites the requirement and reads given / when / then; the requirement check fails if either is missing.

```

43  \(/;
44  expect(decision, "routes translate; put the rule
45  on the " +
46  "entity if it concerns one task, otherwise in
47  the service")
48  .toBeUndefined();

```

Listing 2 Two executable rules with teaching failures in the TypeScript receiver. The second closes a gap that no behavioural test can see (Section 7).

```

49  // Record<ErrorCode, number> is exhaustive: a new
50  error code
51  // will not compile until it has a status.
52  const STATUS_BY_CODE: Record<ErrorCode, number> =
53  {
54  task_not_found: 404,
55  invalid_transition: 409,
56  wip_limit_exceeded: 409,
57  };

```

Listing 3 Unrepresentable Mistake in the TypeScript receiver.

Its architecture rules are the cheapest of the three styles, regular expressions over source text, and the least precise. They are enough for a codebase this small. A larger one would need a parser, and Section 5 shows what a production TypeScript codebase used instead.

4.4. Equivalence in Java and Python

Table 4 shows that the Java and Python receivers bind every rung pattern the TypeScript receiver binds. Reading a row shows one pattern bound three ways; reading a column shows one receiver's complete path. The behaviour is identical: the same ten requirements, the same scenarios, and the same fourteen seeded mistakes caught (Section 7). What differs is the rung on which a language can carry a rule.

Java The Java receiver reaches the top of the ladder most often. Its domain errors form a sealed hierarchy, and the handler that maps them to HTTP uses a switch with no default branch, so the TypeScript Record has an exact equivalent at L5. Its architecture rules are ArchUnit rules over bytecode, the most precise of the three styles, and each carries a because clause that ArchUnit prints on failure; its rule for side effects, the counterpart of the TypeScript console entry, reads "side effects (logs, mail, audit) listen for a domain event; publish one

Table 4 Binding paths of the three receivers. TypeScript is the worked example; Java and Python show equivalence.

Rung pattern	TypeScript (Fastify)	Java (Spring Boot)	Python (FastAPI)
WORKED SLICE	<code>moveTask</code> , service to route	<code>moveTask</code> , service to controller	<code>move_task</code> , service to route
SHORT MANIFEST	AGENTS.md, 468 words	AGENTS.md, 480 words	AGENTS.md, 427 words
EXECUTABLE RULE	source text, 5 rules	ArchUnit over bytecode, 8 rules	AST walk, 7 rules incl. strict type check
CLOSURE CHECK	errors by the compiler; requirements by a test	errors by the compiler; requirements by <code>RequirementsTest</code>	errors and requirements by tests
HONEST DOUBLE	contract run on SQLite and in-memory	contract run on JDBC and in-memory	contract run on SQLite and in-memory
TEACHING FAILURE	a message on every assertion	<code>because(...)</code> on every rule	a message on every assertion
UNREPRESENTABLE MISTAKE	<code>Record<ErrorCode, number></code>	sealed <code>DomainException</code> , exhaustive <code>switch</code>	not available; bound at L3
SINGLE GATE	<code>npm test</code>	<code>mvn verify</code>	<code>pytest</code>
HUMAN BOUNDARY	the same three items in each manifest: dependencies; the rules and manifest; schema and endpoint shape		

instead”. Its requirement check imports the test classes themselves and asserts that every scenario’s `@DisplayName` opens with a requirement id, where TypeScript reads test titles.

Python Python cannot make an unmapped error unrepresentable, so the Python receiver binds that rule one rung lower: a CLOSURE CHECK asserts that every subclass of `DomainError` has an entry in the status map. It compensates elsewhere. Its architecture rules walk the syntax tree, a stricter version of the TypeScript text rules: the rule that a route decides nothing is checked against `If`, `loop`, `comprehension`, `Raise` and `Try` nodes rather than keywords. The strict type checker runs inside the same gate. Scenarios cite requirements in docstrings where TypeScript uses titles.

4.5. Layered, not Clean, and what that costs

All three receivers are layered rather than hexagonal or Clean. The persistence interface lives beside its implementation in the repositories layer rather than as a port owned by the core, so dependency inversion does not stop a service from reaching SQL. Capability confinement does: a rule that only the repositories layer may use the database driver, only the api layer the web framework, only the config layer the environment, and only the events layer logging. We read this as a finding about the ladder rather than about architecture styles. A handful of EXECUTABLE RULES recovers most of the domain protection Clean Architecture provides, without the ports, adapters and interactors an agent would otherwise have to learn and reproduce in every feature. The trade is ceremony against rules, and the ladder lets a team make it in the open.

4.6. What each receiver gains

Working backwards from the receivers, each gains the same four things. The domain stays in one place, which the seeding results in Section 7 measure directly. A new feature has a recipe: state the rule in EARS, let its pattern choose the layer, copy the

slice, run the gate. Failures say what to do. And the manifest a new contributor reads is under 500 words, because everything checkable lives in the checks. The receivers differ only in which rung carries which rule, and the differences are worth reading: the same rule set, bound by three languages, lands on three different sets of rungs.

5. Retrofitting a production codebase: Tutors

The receivers of Section 4 were built with the ladder in mind. This section asks whether the ladder can be climbed by a codebase that was not: an open-source teaching platform in production, with years of history and few tests, into which coding agents were already contributing. It reports what was retrofitted, in what order, what it reached and what it could not, and how its release gate was extracted into a separate, reusable repository. Every figure below is read from the two public repositories as they stood on 28 September 2026: the monorepo ([Tutors contributors 2026a](#)) and the release harness at version 1.15.0 ([Tutors contributors 2026b](#)). Where a figure is our own count rather than one the repositories record, we say so.

5.1. Context

Tutors is a course reader and learning platform. Lecturers write courses in Markdown; a generator turns a course folder into one `tutors.json`; four SvelteKit applications (reader, catalogue, live and time) render it, track presence and report learning time. The stack is TypeScript, Svelte 5 and SvelteKit, six Deno packages published to JSR, pnpm workspaces and Supabase (README.md, ARCHITECTURE.md). The monorepo holds the four applications and 24 packages. By our count, product code in `apps/` and `packages/` is about 24,500 lines of TypeScript and Svelte, excluding tests, fixtures, declaration files and a vendored PDF worker; test code is about 33,500 lines.

1 The monorepo was founded on 27 July 2026 by consolidating
2 several repositories; the original reader repository has history
3 from 2022 (guides/evolution/tutors-revolution
4 .md). On 1 June 2026 it had no tests, no CI workflow and no
5 Dockerfile, and the whole ecosystem had 27 tests (guides/
6 testing/journey.md). Coding agents took part in the
7 retrofit throughout: by our count of git trailers, 329 of the
8 monorepo's 483 non-merge commits, and all 144 of the har-
9 ness's, were authored or co-authored by an agent. The retrofit
10 is therefore a second known use of the paper's setting, not a
11 laboratory one: a domain protected while agents change it.

12 5.2. The retrofit, rung by rung

13 Table 5 maps each rung pattern to the artefact that binds it in
14 Tutors, and Figure 4 dates the retrofit beside the receivers' own
15 iterations. We give dates rather than commits, because what
16 matters is the maturity each point marks and not the hash that
17 records it; the hashes are in the repositories for anyone who
18 wants them. The order says something about what each rung
19 costs. The first moves, in early August, replaced call-sequence
20 mocks with real implementations and a data-driven stand-in
21 carrying 27 validation tests of its own, and cleared 138 type
22 errors. These are the cheapest rungs: they make existing tests
23 honest before new ones are added. Structure came next. In mid
24 September a "testing runway" added architecture rules, suite-
25 health lints and completeness checks as tiers A to O, and one
26 aggregate CI job, ci-success, as the required status. Require-
27 ments came last. Feature files had existed since August, but
28 on 17 September all 24 were found to be documentation only;
29 binding them to product code found bugs in search, sort and log-
30 ging (guides/evolution/tutors-revolution.md, "Spec-first"). EARS Rule: blocks with permanent ids and an
31 audit followed on 21 September, and on 26 September coverage
32 and mutation floors.
33

34 **Structure** The architecture rules state Tutors' layering (foun-
35 dation, core, feature, three UI layers, apps) as one list in
36 .dependency-cruiser.cjs, from which one rule per layer is
37 generated. Each rule carries a comment that says why and
38 what to do instead, and each has a negative fixture under
39 tests/architecture/fixtures/, a small workspace
40 that breaks it on purpose. The test asserts that every fix-
41 ture is caught, that no clean file is flagged, and that the
42 live repository adds no violation beyond a baseline (tests/
43 architecture/dependency-rules.test.ts). Tu-
44 tors is layered, like the receivers, and enforces the layering
45 by the same move: an EXECUTABLE RULE rather than ports
46 and adapters.

47 **Behaviour and requirements** Behaviour and requirements
48 are carried together. A requirement is the title of a Gherkin
49 Rule:, written in EARS (Mavin et al. 2009) with exactly one
50 "shall", a named system and a permanent id (@rule-0110);
51 its scenarios are the proof, bound to product code by
52 vitest-cucumber. Listing 4 shows one. The audit is
53 a CLOSURE CHECK in both directions, and names each
54 failure with a code and its fix (.claude/skills/ears
55 -gherkin-dev/references/audit-codes.md). It

reaches past the repository: a release claim cites a Rule id, and
pnpm check:release-claims fails when no feature defines
it (guides/testing/EARS-METHODOLOGY.md). A re-
quirement is thus traced from its sentence to its scenario, its
changelog line and the observable difference it licenses. Un-
like the receivers, Tutors files Rules by persona rather than by
layer, so it does not use the EARS pattern as a placement oracle
(Table 3).

```
1 @rule-0111 @ears-unwanted
2 Rule: If total or per-package test coverage falls
   below
3   its recorded floor, then tutors shall fail the
   coverage
4   check and name the scope and metric.
5   Scenario: A package below its floor fails the
   check
6   Given recorded coverage floors where
7     "packages/jsr/model/src/**" holds
   statements at 83 percent
8   When the measured statement coverage of
9     "packages/jsr/model/src" is 82.9 percent
10  Then the coverage check shall report "below-
   floor:
11    packages/jsr/model/src/** statements 82.9%
   < floor 83%"
```

Listing 4 An EARS Rule in Tutors, from the test-quality
ratchets feature file. The audit checks its form and tag; its
scenarios check its truth. Steps are wrapped for width.

Climbing in steps Two rules show the ladder being climbed
deliberately rather than all at once. Type checks ran with
continue-on-error until the type errors were cleared, and
became blocking on 18 September. The EARS audit was added
as a non-blocking job on 21 September and joined the required
ci-success job two days later. In both cases the rule was
declared and measured before it was allowed to stop the line.

5.3. What a retrofit adds: ratchets and self-tests

A greenfield receiver can make a rule true everywhere on the
day it is written. A retrofit cannot. Tutors' answer to that is the
part of the retrofit we would most like other teams to copy. Nine
baseline files list what is known to be wrong when a check is
introduced. The check fails on any violation not in the baseline
and on any baseline entry that no longer occurs, so the list can
only shrink (scripts/checks/lib/ratchet.ts). The
nine held 169 entries on 21 September and 167 on 28 Septem-
ber (journey.md). Coverage and mutation floors ratchet
the other way: each is the measured value rounded down,
and a floor two or more points below the measurement fails
as stale (tests/suite-health/coverage-floors
.json, tests/mutation/mutation-floors.json).
A ratchet lets a rule sit at L3 for new code on the day it arrives
while old code is paid down from a ledger. The receivers never
needed this, and a codebase with history cannot do without it.

The second addition answers the paper's own evaluation
method. Section 7 seeded mistakes from outside the receivers
to show the checks can fail. Tutors keeps that evidence in-
side the repository. "No tier without a negative fixture" is a
stated rule (journey.md). Mutation testing covers twelve tar-
geted modules, breaking below 90% with a measured score of

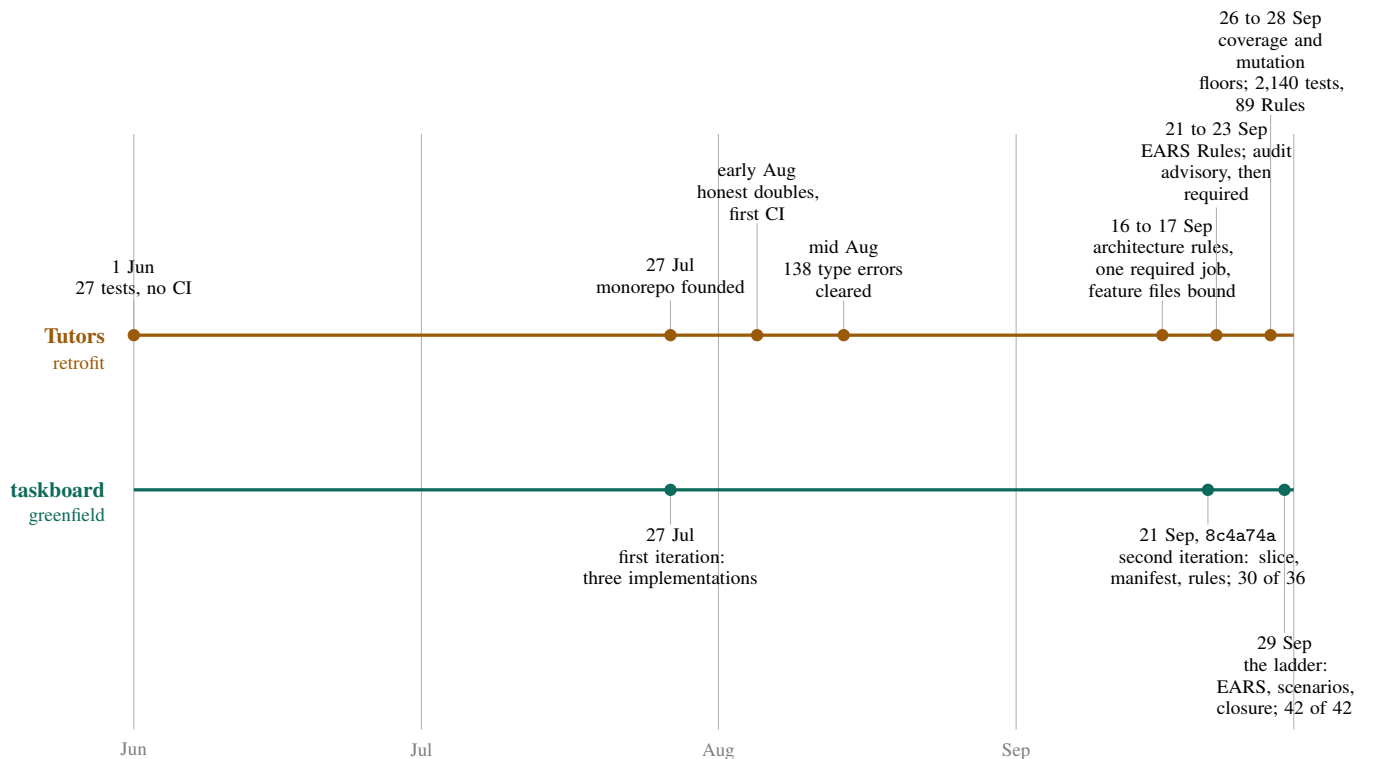


Figure 4 Two climbs, June to September 2026. Tutors, a production codebase, and the receivers, built for the paper, reached the rungs over the same weeks. Each point is a level of maturity; the one commit named, 8c4a74a, is kept because the evaluation files are named by it.

93.6%, and a nightly run over every library module, whose first baseline scored 58.5% over 95 modules (guides/testing/MUTATION-TESTING.md). The harness applies the same rule to itself: a new engine does not merge without an A/A test, a planted change it catches and a change it must not flag (harness docs/user-guide/09-extending.md).

5.4. Abstraction: the release harness

The last rung Tutors added sits outside the monorepo (Figure 5). Its precursor was an in-repository check that the generator’s output matched production’s, byte for byte (tests/release/RELEASE-TESTING.md). The release harness generalises that check to everything a user can observe. It runs a candidate’s images beside production’s, drives the same scripted journeys through both, captures nineteen kinds of artefact (DOM, screenshots, network, console, headers, accessibility, focus order, timing, metrics, logs, persisted writes, SBOM, vulnerabilities, container posture and others), masks known noise, and fails unless every difference is covered by a claim (harness README.md). A claim names an artefact, a scope and the Rule or changelog entry that justifies it (release/claims.yaml). This is a CLOSURE CHECK over the release: every observable difference has a claim, and a claim that matches nothing is reported as stale.

Why a separate repository? The reason the harness gives is the one we gave in Section 3: a check that an agent can edit is a check that an agent can weaken. The harness “consumes the monorepo’s published images . . . and knows nothing about their source, so it can compare any two tags . . . and cannot be quietly

weakened by the PR it is judging” (harness README.md; decision 1 in docs/architecture/06-decisions.md). The receivers bind the HUMAN BOUNDARY at L2, as a line in the manifest. Extraction binds part of it structurally: the gate’s code, its masks and its mutants live where the change under review cannot reach them, behind their own CODEOWNERS. The harness also holds itself to the discipline it imposes. It may fail a release only after a clean production-against-production run less than seven days old, and it must catch ten planted regressions, each attributed to the expected artefact, weekly and on any change to an engine (harness mutants/mutants.yaml, src/gate.ts).

What is generic is the contract, not the stack. The integration contract is versioned and “is what tutors-sdk/tutors-mono-repo (or anything else) may build against”; a test fails when document and code drift (harness docs/contract.md). An adopter supplies signed images, a claims file and optionally a rules.json of Rule ids, and triggers the harness by dispatch (harness docs/monorepo/README.md). The four application names, the journeys and the fixture stubs are still Tutors’. We therefore read the harness as the paradigm made reusable at the level of its contract, and not yet as a tool demonstrated on a second product.

Lean, named in the code The harness is explicit about its Lean vocabulary, and each term points to a mechanism rather than sitting in a glossary (harness docs/lean.md; release/SOP.md). *Jidoka*: the Gate stops the line, and an

Table 5 The rung patterns as bound in Tutors, with the reach of each. Paths are in the monorepo unless marked *harness*.

Rung pattern	Tutors artefact	Reach
WORKED SLICE (L1)	docs/COURSE-PAGE-WALKTHROUGH.md: one URL followed through every file to the rendered cards	a slice in prose, not a small runnable exemplar
SHORT MANIFEST (L2)	CONTRIBUTING.md (2,043 words), ARCHITECTURE.md; the ears-gherkin-dev agent skill (962 words)	no AGENTS.md or CLAUDE.md; the skill loads on demand, not every session
EXECUTABLE RULE (L3)	.dependency-cruiser.cjs: ten rules (six layer rules, no app-to-app, no relative cross-workspace import, primitives off data packages, no cross-package cycle); knip; manifest parity	new code at L3; three older violations held in known-violations.txt
CLOSURE CHECK (L3)	EARS audit, scripts/checks/ears-audit.ts: every Rule has a scenario, no scenario outside a Rule, every Rule bound, every UI scenario has a Playwright test and the reverse; every claim cites a Rule that exists	95 Rules, 193 scenarios in 34 feature files (our count); audit required in ci-success since 23 Sep
HONEST DOUBLE (L3)	MockSupabaseClient with its own tests; <i>harness</i> : a second, in-memory backend behind each seam, driving the same collector	data-driven, not one contract run on both sides
TEACHING FAILURE (L4)	a comment on every dependency rule; ratchet messages (“Fix them rather than adding them to ...”); an audit code table with the fix per code	L4 wherever a check exists
UNREPRESENTABLE MISTAKE (L5)	strict TypeScript and blocking svelte-check; Zod validation of the environment	weak: LoTypes is typed string[], so LoType is string
SINGLE GATE	locally pnpm lint, pnpm test, pnpm check; in CI one required job over eleven; at release, the harness Gate	three commands, then one status
HUMAN BOUNDARY	CODEOWNERS on release/claims.yaml and release/SOP.md; <i>harness</i> : CODEOWNERS on masks, mutants, gate.ts and the contract; a broad claim needs a named person	declared; review not yet practised

advisory confidence score may never override it (“a FAIL is a FAIL at RCS 99”). *Standard work*: a twelve-step release SOP with an owner, an input and a done-when per step, in which steps 2 to 7 run as one command that stops before step 8, the one step that must stay human. *Gemba*: a reviewer’s glance of at most seven ranked places, each marked verified, disputed or escalated. *Kaizen* and root cause analysis: *harness* why opens a 5 Whys whose first answer is the harness’s own trace, rejects “human error” as an answer, and requires one of seven countermeasure kinds, a new mutant being preferred. *Value stream*: *harness* a3 draws an A3 whose current condition is a value stream map of a change from merge to production. Compression is written in as well: “six journeys in three sets, not a hundred; ten mutants, not thirty. ... Add a journey only when a real regression escaped that a journey would have caught” (*harness* README.md, “Where to stop”).

5.5. What the retrofit shows

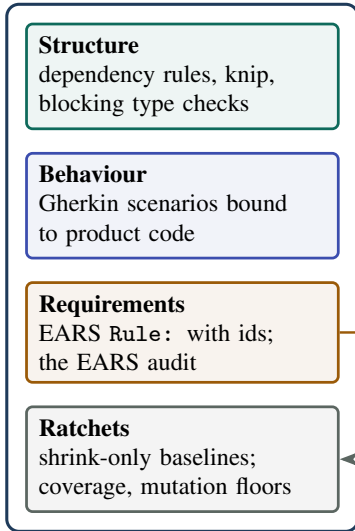
Three results are measured by the repositories themselves, and together they are the agentic loop of Section 3 run at length: agents were trusted with most of the commits, the gate grew under them, and each miss raised a rung. First, the ladder can be climbed on a production codebase in weeks: from 27 tests on 1 June to 2,140 declared tests, 184 bound scenarios under 89 EARS Rules and 51 CI jobs on 28 September (journey.md).

Second, the release rung finds what the lower rungs miss. The harness found four production bugs that no test had, including an authentication page that answered 500 and a first visit that was never recorded (journey.md, “Judge every release against production”). Third, it applies to a real change the same root-cause discipline Section 8 applies to the seeding misses: three 5 Whys are open in the register, each ending in a countermeasure to the system rather than to a person (*harness* kaizen/README.md).

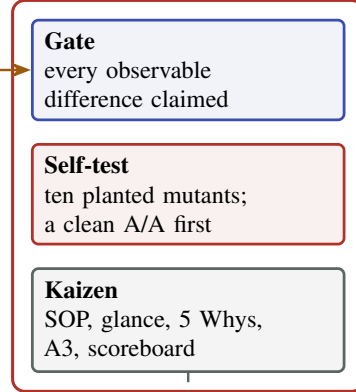
Two results are demonstrated rather than measured. The three kinds of intent survive translation to a codebase the ladder did not shape, and a requirement id can carry intent from a sentence through a scenario to a release decision. And the retrofit adds two things the receivers lack and a greenfield project would benefit from: ratchets for legacy debt, and self-tests that keep the seeding evidence inside the repository.

We narrow the example to TypeScript because Tutors is written in it. The mechanisms have counterparts in the other receivers’ stacks: the dependency rules correspond to ArchUnit and to the Python receiver’s syntax-tree rules, and mutation tools exist for the JVM and for Python. The harness judges images and claims, not source, so we expect a Java or Python service to adopt it through the same contract; that is a hypothesis, not yet tried.

tutors-mono-repo: rungs checked on every PR



tutors-release-harness: the judge, kept apart



a 5 Whys ends in a countermeasure:
a mutant, an EARS spec, an SOP change...

Figure 5 Tutors keeps structure, behaviour and requirement checks in the monorepo and ratchets their debt. The judge of a release is a separate repository that reads only signed images, the claims and the Rule ids, and whose 5 Whys feed countermeasures back into both.

5.6. Limits

The retrofit does not reach every rung, and its own documents say so. There is no short manifest loaded every session; the nearest are a 2,043-word contributing guide and one agent skill loaded on demand. L5 is weak where the domain is richest: the learning-object type that the whole model turns on resolves to `string` (`packages/jsr/model/src/types/type-utils.ts`). The global coverage floor was 58% of statements against a 90% target on 28 September, and tier F, authorisation, is planned but not built (`journey.md`). The HUMAN BOUNDARY is declared but not yet practised: none of the 55 pull requests merged since release 16.2.2 had an approving review, and “the CI gate, the ratchets and the harness stand in for review; they do not replace it” (`journey.md`). The release-confidence tooling is built, but its evidence gates need real releases: the scoreboard is empty (`harness/scoreboard/releases.jsonl`), no 5 Whys is yet closed, and the daily forecast of main against production was FAIL with 892 unclaimed differences on 28 September, most of them an intended image change that nobody had claimed (`guides/evolution/tutors-revolution.md`). Finally, Tutors is one codebase, the retrofit was carried out largely by two of this paper’s authors working with agents, and it was not designed as an experiment. We report it as a known use of the pattern, not as a test of it.

6. Use cases

The receivers show the pattern bound in code. This section states, independently of any language, the situations in which a domain is at risk once agents contribute, so that the pattern can be applied to domains other than task boards. Each use case names a situation, the threat, the invariant at stake and the teaching move that answers it. Table 6 lists twelve in six

families and Figure 6 places each on the ladder, both in the receivers as first published and as they stand after the work described here.

6.1. Placement

Placement failures are the signature of sprawl. In UC1 an agent with no prior session implements a feature end to end; the risk is not that the feature fails but that each layer gets a plausible and inconsistent piece of it. The answer is a worked slice to copy, followed by architecture rules to pass. UC2 and UC3 are subtler because the mistake preserves behaviour. A notification called inline in a service, instead of published as an event, produces the same log line; a work-in-progress check copied into a route produces the same refusal. No behavioural test can tell the difference, and at commit 8c4a74a nothing in any receiver did. They were closed by two EXECUTABLE RULES, that only the events layer may log or notify and that routes make no decisions, each with a TEACHING FAILURE naming where the code should go.

6.2. Contamination

In UC4 a framework annotation looks convenient on a domain object; in UC5 a service needs a setting or a quick query. Both are answered by capability confinement (Section 4), and both were already checked before this work; they gained explained failures.

6.3. Contract integrity

UC6, a new failure mode, is the one use case the receivers answer at L5 in two of three languages: the new error does not compile until it has a status. UC7 is the test double that lies, answered by an HONEST DOUBLE. It has a history: in the first seeding run the contract created tasks titled First, Second and Third, whose alphabetical order equals their creation order, so

Table 6 Twelve use cases in six families. The last two columns give the rung at which the receivers asserted the invariant at commit 8c4a74a and now.

	Situation	Invariant at stake	Then	Now
<i>Placement: sprawl spreads one rule across many places</i>				
UC1	A memoryless agent adds a feature	every part lands in the layer that owns it	L3	L4
UC2	A side effect is added to a use case	use cases publish facts; reactions subscribe	L2	L4
UC3	A business rule drifts to the edge	routes translate; they never decide	L0	L4
<i>Contamination: convenience pulls infrastructure into the core</i>				
UC4	A framework type enters the domain	the domain imports only the standard library	L3	L4
UC5	An outside capability leaks into a service	each capability has one owning layer	L3	L4
<i>Contract integrity: the promises the domain makes</i>				
UC6	A new failure mode is introduced	every error has one public translation	L5	L5
UC7	A test double lies	every double passes the real contract	L3	L3
<i>Harness erosion: the protections themselves wear down</i>				
UC8	The agent edits a rule to go green	rules change only with a person's decision	L2	L2
UC9	A dependency is added	new dependencies need a person	L2	L2
<i>Cognitive load: what people and agents can hold in mind</i>				
UC10	A reviewer faces a large AI diff	intent is reviewable apart from code	L1	L3
UC11	An agent starts cold	the declaration stays small	L3	L3
<i>Transfer: taking the approach to a new domain</i>				
UC12	A new domain is embedded	each invariant is placed on the ladder deliberately	L1	L1

a double that sorted by title could not be caught. Changing the titles to Write, Review and Publish fixed it. We had written a check that could not fail. Correcting the fixture was a small piece of kaizen, and a reminder that a check is only as good as the data it runs on.

6.4. Harness erosion

UC8 and UC9 concern the ladder itself. An agent whose build is red has an easy way to make it green: relax the rule. No agent in our pilot did so, but only the manifest forbade it (L2). Raising UC8 to L3 is a matter of repository settings rather than tests, for example by requiring a named person's review for any change to the rule files. Dependencies (UC9) are similar, and can be checked by pinning the dependency list in a test.

6.5. Cognitive load

UC10 is the reviewer's problem. When a change adds a feature, the parts a person most needs to judge are the new rule and the examples that illustrate it, not the plumbing. Keeping requirements in their own file, with scenarios that cite them and a check that keeps the two in step, puts the intent of a change in a small, separate diff. We claim this lowers the reviewer's load; we have not yet measured it. UC11 is the agent's problem: every session starts cold, and guidance that grows to compensate ends up unread. Fading keeps the manifest under 500 words even after

two new kinds of intent were added.

6.6. Transfer

UC12 asks whether the pattern survives a move to a domain that is not a task board. Two illustrations, not implemented, suggest how. In a double-entry ledger, the invariant that debits equal credits can be stated in EARS as an unwanted behaviour, shown in a posting slice, checked by a property test, and prevented by a transaction type that can only be built from balanced legs. In medication dosing, the invariant that a dose may not exceed a weight-based maximum without a clinician's override can be checked by boundary scenarios and prevented, in part, by types that distinguish milligrams from milligrams per kilogram. In both, the question the ladder asks is the same: how high must this rule climb, given what it costs to get it wrong?

7. Evaluation

We ask three questions. **RQ1:** which mistakes does each rung catch, and which does it miss? **RQ2:** what do the added rungs cost in size and reading? **RQ3:** do the declared and enforced rungs change what coding agents do? Every number below can be regenerated from the repository's evaluation/ directory (Griffin 2026a).

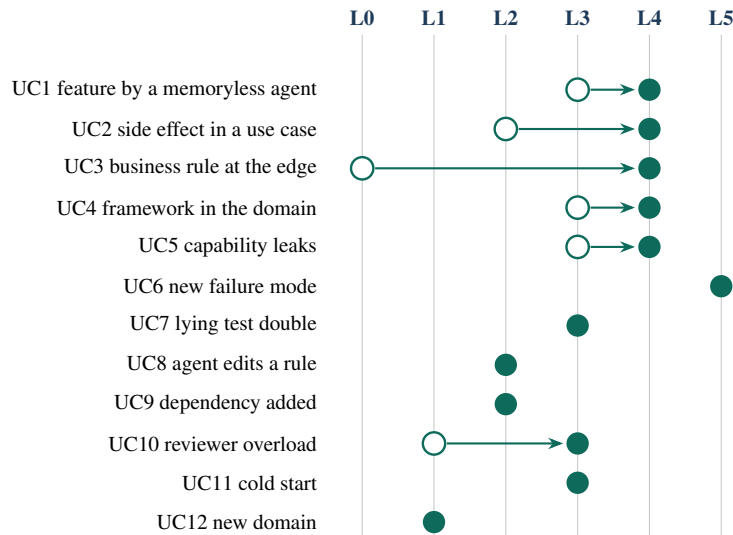


Figure 6 Where each use case sits on the ladder in the receivers, at commit 8c4a74a (hollow) and now (filled). UC6 is at L5 in Java and TypeScript and at L3 in Python.

7.1. RQ1: violation seeding

A seeding script plants one canonical mistake at a time in a receiver, runs its single verify command, records the earliest mechanism that objects, and restores the file. It refuses to start unless the receiver is green, and checks that it is green again at the end. The mechanisms, in order of how early they give feedback, are the compiler, an architecture rule, the requirement check, a behavioural test, and nothing. Table 7 lists the fourteen mistakes: twelve structural, and two that break the trace between requirements and scenarios.

Two states of the receivers are compared throughout. We name the earlier one by the commit that records it, 8c4a74a, because the files in `evaluation/` are named by it, but the hash stands for a point of maturity on the timeline in Figure 4: the second iteration of 21 September 2026, with a slice, a manifest and structural rules and nothing above them. “Now” is the ladder as it stood on 29 September.

Figure 7 compares the receivers at 8c4a74a, before the ladder was added, with the receivers now. Before, the single gate caught 30 of 36 structural mistakes. The six misses were the same two mistakes in every language: V8, a side effect inlined in the service, and V10, a business rule placed in the route. The same two mistakes got through in three languages, past three different styles of enforcement, which told us the gap was in the pattern and not in any one receiver. Both mistakes preserve behaviour, so no behavioural test could have caught them; only a structural rule could. After two such rules were added, and the requirement check with its two seeds, the gate catches all 42 mistakes. The new routes rule also moved V7 from a behavioural test to an architecture rule in Python and TypeScript, which is earlier feedback for the same mistake. Application code did not change between the two runs; only tests, manifests and requirement files did.

7.2. RQ2: cost

Table 8 reports size. Application code is unchanged. Test code grew by between 31 and 89 lines per receiver, and each manifest by between 80 and 83 words, remaining under the 500 words the plan allows. The added rungs therefore cost about one screen of test code and one paragraph of prose per receiver, most of it the requirement check and the ids on existing tests.

7.3. RQ3: a pilot with coding agents

Eighteen coding agents, each starting with no memory, implemented the same feature request, task deletion, in copies of the receivers at commit 8c4a74a. Each copy offered one, two or three of the carriers: the exemplar only (L1), the exemplar and the manifest (L1–L2), or the repository as published (L1–L3). The design crossed three languages, three conditions and two models, with one run per cell. A hidden acceptance test of four cases, never shown to an agent, judged the result, and a scripted audit fixed in advance of inspection recorded where each agent placed each part of the change.

All eighteen runs delivered a passing suite and passed every acceptance case (Table 9). Two departed from the conventions, both by the smaller model in the exemplar-only condition. One, in Java, logged the deletion inside the service instead of publishing an event: the V8 mistake, which no rule caught at the time. The deletion request’s last clause, “whenever a task is deleted the team must be told”, is an event-driven requirement in EARS terms, and the mapping in Table 3 places it in an event handler. The other, in Python, did not add the new operation to the in-memory double, which only the type checker in the full rule set notices. With one run per cell these results are suggestive, not significant. The pilot has not yet been repeated against the receivers as they stand now, with the deletion request written in EARS; that is the next experiment.

Table 7 The fourteen seeded mistakes, and the mechanism that caught each in Java / Python / TypeScript now. C compiler, A architecture rule, R requirement check, B behavioural test.

	Mistake	J / P / T
V1	The API bypasses the service and uses a repository	A / A / A
V2	A service signals failure with an HTTP concept	A / A / B
V3	A service issues SQL	A / A / A
V4	A component constructs its own collaborator	A / A / A
V5	A service reads the environment	A / A / A
V6	A new domain error has no HTTP status	C / A / C
V7	A route catches a domain error and answers ad hoc	B / A / A
V8	A side effect is inlined in the service	A / A / A
V9	The test double drifts from the real repository	B / B / B
V10	A business rule is placed in the route	A / A / A
V11	A setting is hard-coded in the service	B / B / B
V12	A framework type enters the domain	A / A / A
V13	A requirement has no scenario	R / R / R
V14	A scenario cites no requirement	R / R / R

Table 8 Size of the receivers at commit 8c4a74a and now (evaluation/metrics*.json).

Receiver	Application lines		Test lines		Manifest words	
	then	now	then	now	then	now
Java	283	283	287	376	397	480
Python	223	223	193	254	347	427
TypeScript	242	242	193	224	388	468

8. Discussion

8.1. Lean explains the ladder

Lean thinking faced a version of the problem this paper faces: work done quickly by many hands, more of it than anyone could inspect, where a defect that travelled downstream cost far more to fix than one caught where it was made (Ohno 1988; Womack & Jones 1996). Its answer was to build the specification into the work itself. We read the Assertion Ladder as that answer applied to a repository and a memoryless contributor, and this section makes the case through four Lean ideas: value, standard work, the value stream and root cause analysis. Table 10 maps each to the receivers and to the evidence, and the section closes with the places where the analogy stops.

Value Lean asks first what the customer values, and the ladder adds the domain to the answer: a change is valuable only if it works and leaves the domain’s rules in one place. Sprawl is then easy to name as waste. It is work that looks finished but will be redone, and the rework falls on whoever next tries to change the rule. The same lens explains the force most easily forgotten, ease of use. A paragraph of prose that repeats what

a test enforces is read by every contributor in every session and adds nothing the gate does not already guarantee; for a contributor that relearns everything each time, that reading is the dominant cost. Fading removes it. The receivers show both halves: application code fell from 637, 729 and 957 lines to 283, 223 and 242 when containers, mappers and a second entity were removed, and the manifests stayed under 500 words while two new kinds of intent were added.

Standard work Standard work is where the mapping is tightest. The worked slice and the feature recipe of Section 4 (state the rule in EARS, let its pattern choose the layer, copy the slice, run the gate) specify the content of a change, its sequence and its expected outcome, which is how Spear and Bowen describe Toyota’s work (Spear & Bowen 1999). The ladder then does what they found Toyota does: it attaches a test to the standard, so that a departure shows up at once. The lower rungs show and tell the standard; the upper rungs make it self-checking. Every pilot run followed the slice’s layering, and the two departures the pilot recorded both came from the condition with the slice alone and no written standard. Standard work is also owned.

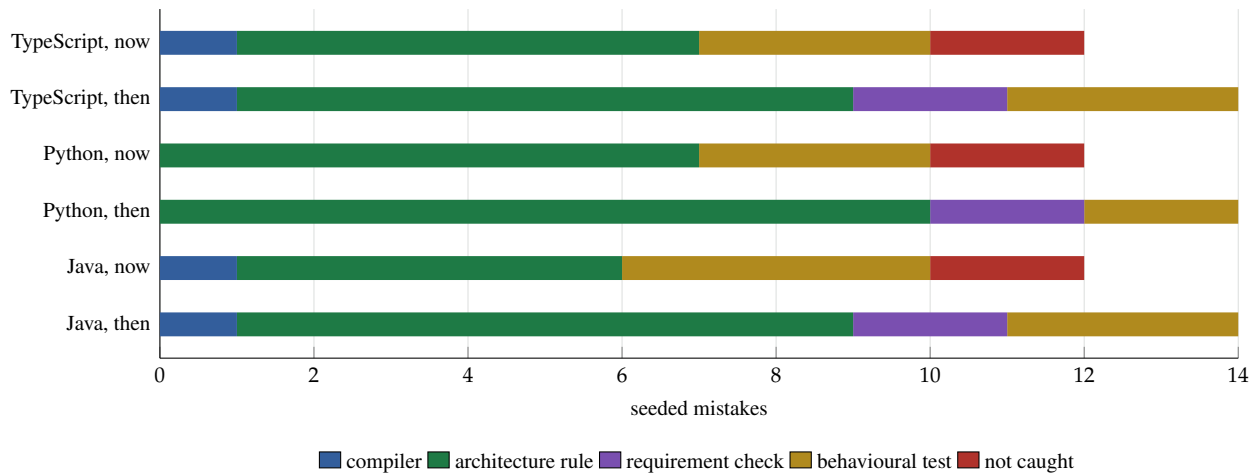


Figure 7 Seeded mistakes by the earliest mechanism that caught them, per receiver, at commit 8c4a74a (12 structural seeds) and now (14 seeds).

Table 9 Pilot results by condition (six runs each; evaluation/agent_experiment/).

Condition	Acceptance passed	Departures from convention	Mean tokens	Mean seconds
Exemplar (L1)	6 / 6	2	61 650	121
+ Manifest (L1–L2)	6 / 6	0	61 990	132
+ Enforced (L1–L3)	6 / 6	0	65 696	151

In Lean, the people who do the work improve the standard; in the receivers, an agent follows the standard, and the HUMAN BOUNDARY requires a person’s decision before the rules or the manifest change. That division is deliberate, and we return to it below.

The value stream Figure 8 draws the value stream of one change: written by an agent, verified by the single gate, reviewed by a person, merged and released. Each rung decides where along that stream a broken rule is found. At L0 only a reviewer who reads closely can see it, and cognitive erosion is the loss of that reading. At L1 and L2 the standard makes the mistake less likely but finds nothing. At L3 and L4 the gate finds it before any person has spent time on the change, and at L5 it cannot be written. The seeding results show the shift. At commit 8c4a74a, V8 and V10 passed the gate in all three receivers and would have reached review; now both are stopped at the gate by an architecture rule. V7 moved earlier within the gate, from a behavioural test to an architecture rule, in Python and TypeScript, and V6 was already refused by the compiler in Java and TypeScript. We have no timing data for the stream, so we claim only the position at which a defect is found, not the time saved. The Lean argument is that position matters: a defect found at the gate goes back to the contributor who made it while the change is still open, whereas one found in review has already cost a person’s attention and, if missed, becomes the rework that sprawl describes.

Root cause analysis The first seeding run is a small worked example of Lean problem solving. The problem was concrete:

six of 36 seeded mistakes passed the gate. Asking why led to a cause at the level of the ladder, not of any receiver. The misses were the same two mistakes, V8 and V10, in all three languages. Both passed because every behavioural test passed; every behavioural test passed because neither mistake changes what the system does; and nothing else looked, because the structural rules covered which layer may import which, not which layer may log, notify or decide. The root cause was that behaviour cannot see placement and no rule asserted placement for side effects or decisions. The countermeasure was two EXECUTABLE RULES, each with a TEACHING FAILURE naming where the code belongs, which is to say two rules moved from L0 and L2 to L4. The check was a second run, which caught all 42 seeds, including the new requirement seeds. The standard then absorbed the countermeasure, in the plan–do–check–act manner of an A3 report (Sobek II & Smalley 2008). The V9 fixture episode (Section 6) followed the same cycle on a smaller scale, and is the kaizen of a check rather than of the code. What Lean adds to the usual advice to add a test after a bug is the insistence on the cause: because the same gap appeared in three languages, the countermeasure was a kind of rule every receiver needed, not a patch to one of them.

Figure 9 writes the episode up as an A3, in the form the Tutors release harness writes for a release (Section 5). We include it because it is the ladder’s hallmark on one page. The current condition is where a mistake was found along the stream, the root cause is the rung it was sitting on, the countermeasure is the rung it climbed to, and the check is the next run. The form carries no verdict of its own, which is the point. In Tutors the

Table 10 Lean ideas, the rungs and patterns that carry them, and where they appear in the receivers. The first four rows are the argument; the last three are supporting mechanisms.

Lean idea	Rung or pattern	In the receivers	Evidence
Value, and waste removed	fading; SHORT MANIFEST	no DI container, no ORM, one entity; no prose for what a test checks	637 / 729 / 957 → 283 / 223 / 242 lines; manifests 480 / 427 / 468 words
Standard work	L1 WORKED SLICE, L2 SHORT MANIFEST, with L3 as its built-in test	the slice and the feature recipe, followed by every contributor and changed only by people	every pilot run followed the slice's layering
Value stream: defects found earlier	L3–L5	V8 and V10 now stopped at the gate, not left to review; V7 moved from a behavioural test to an architecture rule in Python and TypeScript	V8 and V10 passed the gate in all three receivers at 8c4a74a; caught now
Root cause analysis and kaizen	EXECUTABLE RULE, TEACHING FAILURE; HONEST DOUBLE	the misses traced to one cause and closed by two rules; the V9 fixture corrected	30 / 36 → 42 / 42; V9 caught in all three receivers
Jidoka (the line stops itself)	SINGLE GATE	one verify command; “change the code, not the test”	no pilot run modified a rule (18 / 18)
Andon (signal and cord)	L4 TEACHING FAILURE; HUMAN BOUNDARY	failures name the rule and the fix; “ask a human first” for dependencies, rules and schema	message on every rule; boundary declared (L2), not yet enforced
Poka-yoke (mistake-proofing)	L5 UNREPRESENTABLE MISTAKE	exhaustive error mapping that will not compile when a case is missing	V6 caught by the compiler in Java and TypeScript

harness opens an A3 on every failed candidate and its findings inform a confidence score that a person reads before shipping. In the receivers the countermeasure went straight into the gate. A team can use the same sheet for confidence or for a hard check, and the ladder is the scale on which it says which.

Supporting mechanisms Three further Lean mechanisms map onto the ladder more briefly. The single gate is *jidoka*: the build stops itself on an abnormality, so that no person has to watch every line, and the manifest's instruction to change the code rather than the test is the rule that a stop is answered by fixing its cause. No pilot run modified a rule. Jidoka's older name, automation with a human touch, fits a team of people and agents well: the machine detects and stops, the person judges. The TEACHING FAILURE is the *andon* signal, which shows where the problem is and why; the HUMAN BOUNDARY is the andon cord, the right and the duty to stop and ask a person before changing a dependency, a rule or a schema. The UNREPRESENTABLE MISTAKE is *poka-yoke* of the preventive kind.

Where the analogy stops The mapping is close but not exact, and the places where it breaks tell us something. First, an agent does not learn across shifts. A factory worker is taught the standard, practises it and in time no longer needs the sheet; an agent is taught afresh every session. The cost of showing and telling is therefore paid every time, which is the strongest reason to move a rule from prose into a check, where following it costs no reading. For the same reason, improvement cannot

happen inside the agent. Whatever an agent notices in a session is lost unless it is written into the repository, so reflection and kaizen run through people and the files they own. Second, Lean's respect for people becomes the HUMAN BOUNDARY. In Lean the people who do the work own the standard. Here much of the work is done by agents, so people keep what needs judgement: what counts as value, how high a rule should climb, and when a check is wrong. The risk to avoid is the one Lean warned against, reducing people to inspectors of someone else's output; the ladder gives them the standard and its checks to own instead of every line to read. Third, the fixture is made of the same material as the work. A worker cannot redesign the jig in the middle of a shift, but an agent can edit a test, which is why changes to the rules sit behind the HUMAN BOUNDARY, and why use case UC8 remains at L2 until repository settings enforce it. Fourth, the ladder can itself become waste. A check is cheap to run but not free to maintain, and a rule encoded in types far more elaborate than its risk warrants is over-processing. Reinertsen's caution applies (Reinertsen 2009), and so do the findings of Staats and colleagues on uncertain knowledge work (Staats et al. 2011): standardise what should repeat and leave the rest free. The ladder applies Lean to a team's conventions, not to its design.

8.2. Domain sanctity against ease of use

A repository can be made arbitrarily hard to damage by making it arbitrarily hard to change. The ladder resists this in two ways. Each rule climbs only as far as its risk warrants, so the cost of

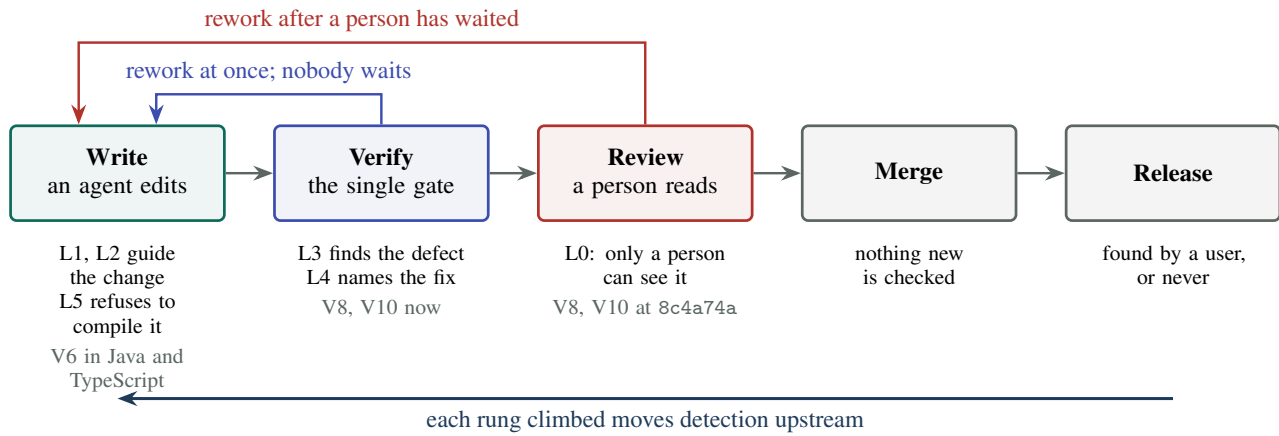


Figure 8 The value stream of one change. Each rung moves the point at which a broken rule is found further upstream. At commit 8c4a74a, V8 and V10 passed the gate and could be seen only in review; they are now stopped at the gate. The figure shows position only; we have no timing data.

a rung is paid only where it buys something. Lean would call the alternative over-processing. And the ladder is compressed from below as it grows from above: when the receivers gained a requirement file, a scenario convention and four new rules, their manifests grew by one paragraph each and stayed under 500 words, so the written standard stayed short enough to be read in every session. The evidence that ease of use survived is modest but concrete: in the pilot, agents given the full rule set finished in 151 seconds on average against 121 for the exemplar alone, and used about 6% more tokens, for a condition in which none departed from the conventions.

8.3. Where each kind of intent stops

Each kind of intent sees some mistakes and is blind to others, which is why the three are complementary rather than redundant. Behaviour cannot see placement: V8 and V10 pass every scenario because what the system does is unchanged. Structure cannot see meaning: an architecture rule will not notice that a task may now skip a step. Requirements cannot execute: without the closure check that ties them to scenarios, they decay into prose, the failure the manifest's own design avoids. The ladder's contribution is to put all three on the same scale, so that a team can ask of any rule, in any of the three, how strongly it is asserted.

8.4. Threats to validity

Construct Seeded mistakes are canonical, not observed; real agents may make mistakes the fourteen do not represent. The TypeScript architecture rules match source text, not syntax, and could be evaded by code written to avoid them.

Internal The authors built the receivers, the seeds and the rules together with an AI coding agent, in the collaborative mode the paper studies. Rules were added after the misses they close were known; the second seeding run therefore shows that the gap can be closed, not that the rules generalise to unseen mistakes.

External One small domain, three mainstream stacks and two models from one vendor limit generality. The pilot ran one agent per cell and has not yet been repeated against the current receivers.

Conclusion Seeding results are deterministic and reproducible from the repository. Pilot results are suggestive only.

9. Related work

The Assertion Ladder draws on six bodies of work that seldom cite one another: architecture conformance, behaviour and requirement specification, studies of coding agents, constraints on generated code, the psychology of learning, and the pattern form. Lean production is treated in Section 8. This section reviews each body of work and says how the ladder builds on it. Each theme ends with the gap in current practice that the ladder proposes to standardise. The gaps are collected at the end as G1 to G5.

9.1. Architecture conformance, erosion and drift

The gap between intended and implemented architecture has been studied for more than thirty years. Perry and Wolf separated *drift*, a growing insensitivity to the architecture, from *erosion*, outright violation of it (D. E. Perry & Wolf 1992). Lehman's laws state that a changing system grows more complex unless work is done to stop it (Lehman 1980). The debt metaphor gives that work a price: shortcuts are borrowed against future change and repaid with interest (Cunningham 1992; Kruchten et al. 2012). Two surveys organise the response. De Silva and Balasubramaniam classify erosion-control techniques into those that minimise, prevent or repair erosion (de Silva & Balasubramaniam 2012). Li et al. map the symptoms, causes and consequences of erosion reported across the literature (R. Li et al. 2022).

Conformance checking is the best-studied remedy. Reflexion models compare a high-level model that an engineer proposes with a model extracted from the source, and report convergences,

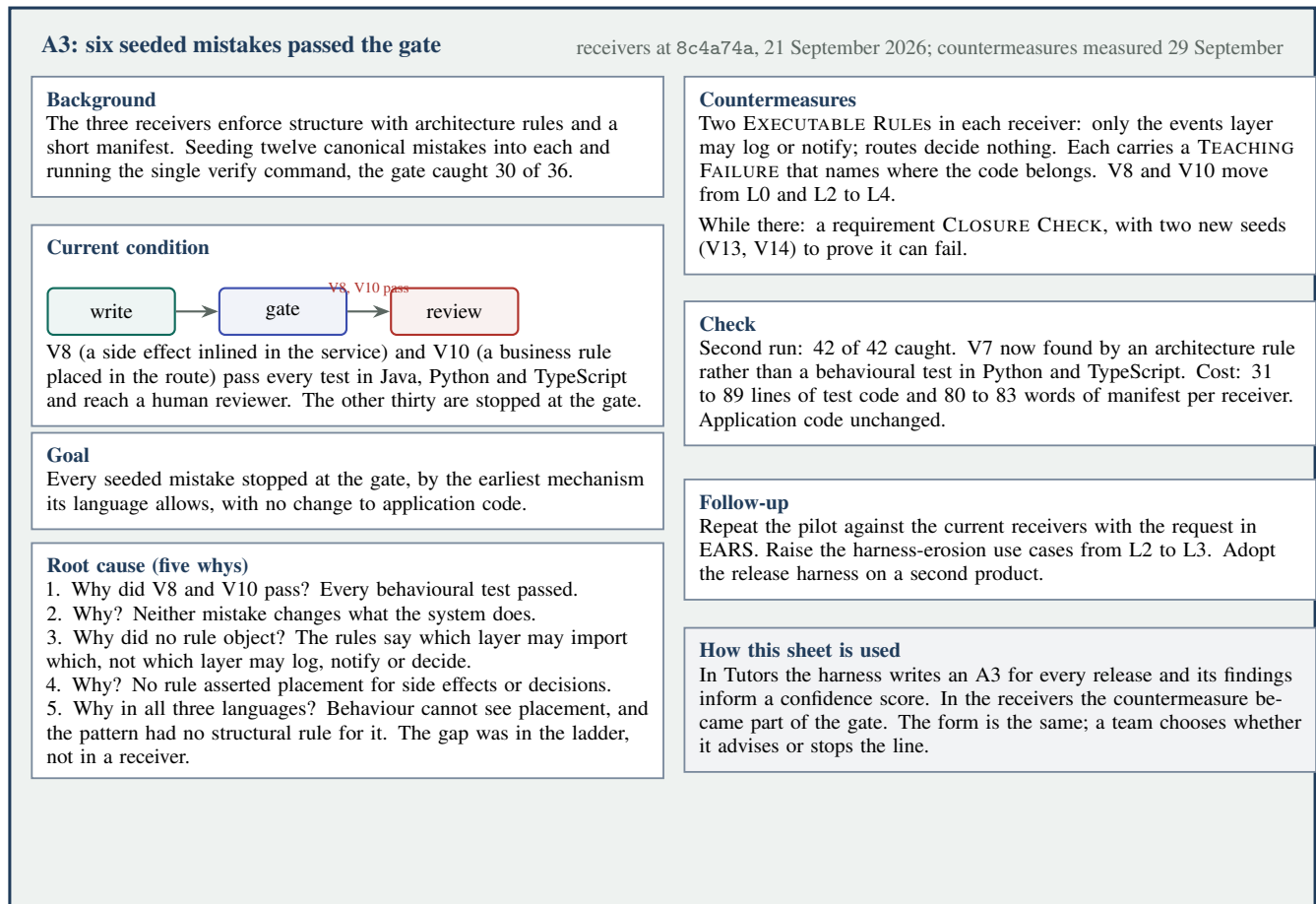


Figure 9 The first seeding run as an A3 report. One sheet carries the problem, where it was found, why, what rung the rule climbed to and what the next run showed. Tutors writes the same sheet for a release and reads it as confidence; the receivers made its countermeasure a gate.

divergences and absences (Murphy et al. 2001). Dependency constraint languages state the allowed and forbidden dependencies between modules and check them statically (Terra & Valente 2009). Passos et al. compare these and related approaches on one system (Passos et al. 2010). Practice has since moved the checks into the build. Architecture fitness functions make an architectural property executable (Ford et al. 2017). ArchUnit expresses dependency rules for Java as ordinary unit tests (TNG Technology Consulting n.d.), Import Linter enforces layer and independence contracts between Python modules (Seddon n.d.), and dependency-cruiser validates JavaScript and TypeScript imports against rules in a configuration file (dependency-cruiser: Validate and Visualise Dependencies, with Your Rules n.d.).

The receivers use this tradition directly: the EXECUTABLE RULE at L3 is a conformance check. The ladder adds two things the tradition does not address. It asks which rules deserve a check at all, since a check has a cost and some rules are better shown or told. It also asks that a failing check teach the fix, which matters when the reader of the failure is an agent that will act on it immediately. The gap is a principle for choosing which rules to check and which to document, and a requirement that a failing check explain itself (G3).

9.2. Specification by example, BDD and requirements

Behaviour-driven development grew out of test-driven development (Beck 2003). It names tests as behaviours in the language of the domain and writes them as given, when and then (North 2006). Specification by example treats a set of such examples as the living documentation of a system (Adzic 2011), and Gherkin with Cucumber makes them executable as plain-text scenarios (Wynne & Hellesøy 2012). Solis and Wang identified the characteristics that define BDD, among them a ubiquitous language, plain-text scenario templates and automated acceptance tests (Solis & Wang 2011). The research record is uneven. Binamungu et al. found that practitioners struggle to keep large scenario suites consistent and free of duplication as they grow (Binamungu et al. 2018). A later mapping study of 166 papers found little industrial evaluation and few metrics for the quality of BDD specifications (Binamungu & Maro 2023).

EARS constrains natural-language requirements to a few templates in order to remove ambiguity (Mavin et al. 2009). Its authors report ten years of adoption across industrial domains (Mavin & Wilkinson 2019). Requirements traceability has a longer history. Gotel and Finkelstein showed that most traceability problems arise before the requirement is written, in the loss of its origin and rationale (Gotel & Finkelstein 1994).

Bjarnason et al. studied six companies and reported recurring challenges in keeping requirements aligned with testing, along with the practices that address them (Bjarnason et al. 2014).

The receivers take the scenario form but not the Gherkin runner. Scenarios are ordinary tests, named in given, when and then, that cite a requirement id. The ladder then adds what the literature treats as an open maintenance problem: a CLOSURE CHECK that fails the build when a requirement has no scenario or a scenario cites no requirement. It also uses the EARS keyword as a hint about placement, a use its designers did not intend. The gap is an executable closure between requirements and scenarios in repositories that agents edit, so that traceability is checked on every change rather than audited now and then (G4).

9.3. Coding agents, their harnesses and their context files

Code generation moved from completing functions (M. Chen et al. 2021) to resolving real issues (Jimenez et al. 2024). The step depended on the harness. SWE-agent showed that an agent-computer interface designed for the model changes what the model can achieve (Yang et al. 2024). AutoCodeRover added code search over the syntax tree and test-based fault localisation (Zhang et al. 2024), and OpenHands provides an open platform for such agents (Wang et al. 2025). These benchmarks and systems measure the agent. The repository is held fixed and treated as the problem to be solved.

Empirical studies of assisted programming give a mixed picture. A controlled experiment found developers with Copilot finished a task 55.8% faster (Peng et al. 2023). A randomised trial with experienced open-source maintainers found that AI tools made them 19% slower (J. Becker et al. 2025). The correctness of suggestions varies by language (Nguyen & Nadi 2022), and security studies show that insecure code passes unnoticed (Pearce et al. 2022; N. Perry et al. 2023). Studies of the process describe a shift from writing to reviewing (Vaithilingam et al. 2022; Barke et al. 2023). At the scale of repositories, an industry report found rising code churn and copy-pasted code and falling moved code as assistants spread (Harding & Kloster 2024). A difference-in-differences study of projects that adopted Cursor found a short rise in velocity and a lasting rise in static-analysis warnings and complexity (He et al. 2026). In a dataset of 456,000 pull requests written by agents, those pull requests were accepted less often than human ones but reviewed much faster (H. Li et al. 2025). These are the conditions under which sprawl and cognitive erosion arise.

Agent context files are the newest object of study. AGENTS.md proposes one Markdown file per repository, read at the start of every session (AGENTS.md: A Simple, Open Format for Guiding Coding Agents n.d.). Studies of such files find shallow structure and mostly operational content (Chatlatanagulchai, Thonglek, et al. 2025), no settled structure across 466 open-source projects (Mohsenimofidi et al. 2026), and, across 2,303 files, frequent guidance on testing and architecture but little on security or performance (Chatlatanagulchai, Li, et al. 2025). Evidence on their effect is mixed. One study associates AGENTS.md files with shorter agent runtimes and fewer to-

kens at the same completion rate (Lulla et al. 2026). Another finds that context files do not generally improve task success and raise inference cost by over 20% (Gloaguen et al. 2026). Work on generated tests shows that models can write useful unit tests (Schäfer et al. 2024), and a review of LLMs in software architecture finds few studies of architecture-aware generation (Schmid et al. 2025).

The ladder places a context file at L2, one rung among six, and treats its growth as a cost. The mixed evidence on context files is what we would expect. Prose that repeats what a test enforces adds cost without adding protection, and prose that states what no test enforces is easily ignored. The receivers keep the manifest short by removing a rule from it once a check covers it. The gap is a link between the instruction file and the checks, so that each told rule is either enforced or deliberately left to judgement (G2). A second gap follows from the benchmarks. SWE-bench and its successors measure agents against fixed repositories. Nobody yet measures the repository, asking whether its harness catches a known set of canonical mistakes whoever makes them (G5). Violation seeding (Section 7) is a first form of such a measure.

9.4. Guardrails and constraints on generated code

A second line of work constrains the model rather than the repository. Constrained decoding restricts generation to programs that parse or type-check. Synchromesh enforces grammar and semantic constraints during decoding (Poesia et al. 2022). Monitor-guided decoding consults a static analysis of the repository so that generated code refers only to names that exist (Agrawal et al. 2023). Type-constrained decoding for TypeScript more than halves compilation errors (Mündler et al. 2025). Other work places feedback in a loop after generation: models improve when they see execution results and explain their own code (X. Chen et al. 2024). Tests can also serve as the specification. Supplying tests with a problem statement raises the rate at which models solve it (Mathews & Nagappan 2024), and generating tests first to clarify intent helps both the model and the programmer who checks its output (Fakhoury et al. 2024).

These techniques sit at particular rungs. Constrained decoding is L5 applied at the model, and feedback loops are L3 and L4 applied at the harness. The ladder places the same assertions in the repository, where they survive a change of model, vendor or harness and bind people as well as agents. The two approaches are complementary. A repository whose rules are executable gives a feedback loop something to run, and a type-level assertion gives constrained decoding something to enforce. What the field lacks is a shared vocabulary for how strongly a rule is asserted, so that a team can say which of its rules are shown, told, checked, explained or prevented, whatever tools do the asserting (G1).

9.5. Pedagogy and cognitive load in software engineering

Cognitive load theory holds that working memory is small and that instruction should not waste it (Sweller 1988). Worked examples are one of its most robust findings (Sweller & Cooper

1985), and minimal guidance is less effective than direct instruction for people new to a domain (Kirschner et al. 2006). Scaffolding gives support that is later withdrawn (Wood et al. 1976), and fading turns worked examples gradually into problems (Renkl & Atkinson 2003). The expertise reversal effect warns that guidance which helps a novice can hinder an expert (Kalyuga et al. 2003). Formative feedback is most useful when it is specific and timely (Shute 2008), and programming error messages have long been criticised for failing to say what to do (B. A. Becker et al. 2019).

Software engineering has studied the same limits. Program comprehension research shows how much of development is spent understanding code rather than writing it (Siegmund 2016), and a mapping study catalogues how the cognitive load of developers is measured (Gonçalves et al. 2019). In code review, understanding the change is the main challenge (Bacchelli & Bird 2013), and working memory capacity is associated with review effectiveness (Baum et al. 2019).

The ladder applies these findings to a new kind of learner. An agent is expert in general and a novice in the particular repository, so the ladder gives it direct instruction and worked examples for the particular (L1, L2) and fades the prose once a check exists, as the expertise reversal effect would advise. TEACHING FAILURE is formative feedback written into the checks. The expertise reversal effect also cuts the other way: an agent that is expert in general will build past what a student can read, so the rungs that teach the agent the particular are also what keep a student's work within their reach, and a team's within its understanding. The reviewer's load is reduced by moving rules out of the diff and into the gate. The same gap appears again: a shared scale on which these teaching moves are named and graded, so that repository design can draw on instructional design on purpose (G1).

9.6. Patterns and pattern languages

A pattern names a problem in a context, the forces that act on it and a resolution (Alexander et al. 1977). The form reached software through the Gang of Four (Gamma et al. 1994) and the POSA series (Buschmann et al. 1996), whose fifth volume examines how patterns combine into sequences and languages (Buschmann et al. 2007). Enterprise application patterns catalogue layering and the service layer used by the receivers (Fowler 2002). Test patterns are closest to the rung patterns. Meszaros catalogues test doubles, including the fake object that stands in for a real dependency (Meszaros 2007). HONEST DOUBLE adds a contract that both the fake and the real implementation must pass.

The Assertion Ladder is a pattern in this classic form with nine smaller patterns beneath it. Its forces are specific to repositories shared with agents: memoryless contributors, review that cannot keep pace, and the cost of every assertion. Existing catalogues name the solutions the rung patterns use, but none grades them against one another.

9.7. From conventions to a ladder

The receivers began, at commit 8c4a74a, with a repository in which conventions are exemplified by one small vertical slice,

declared in a short manifest and enforced by architecture rules written as tests. In the vocabulary of this paper, that arrangement binds L1, L2 and L3 for structure alone, and it is the baseline of our evaluation. The ladder generalises it in three ways. It adds the rungs above and below (L0, L4 and L5) and so grades assertion by strength. It extends the same rungs from structure to behaviour and requirements. And it is measured by a second seeding run with fourteen canonical mistakes rather than twelve, which closes the two misses that the first run found in every language.

Gaps The review leaves five gaps in current practice. The Assertion Ladder proposes a standard for each.

G1 There is no shared vocabulary for how strongly a rule is asserted.

G2 Agent instruction files are unvalidated prose with no link to enforcement.

G3 Conformance tools do not say which rules to check rather than document, or require a failure to name its fix.

G4 Agent-facing repositories have no executable closure between requirements and scenarios.

G5 No benchmark asks whether a repository's harness catches canonical mistakes; SWE-bench measures agents, not repositories.

10. Conclusion

Coding agents make a domain's rules easier to break and harder to notice breaking. The Assertion Ladder answers by asking one question of every rule: how strongly does the repository assert it? Shown, told, checked, explained or prevented. Structure, behaviour and requirements are three loads on the same ladder, and the pattern of an EARS requirement suggests where its code belongs. A greenfield receiver in TypeScript, with equivalent receivers in Java and Python, binds every rung in its language's idiom. Seeding fourteen canonical mistakes into each, the single verify command catches all 42, where it caught 30 of 36 structural mistakes before the behavioural and requirement rungs and two new structural rules were added. The cost was about a screen of tests and a paragraph of prose per receiver. The Tutors retrofit shows the same rungs established in a production code-base that agents were already changing. It reached the checked and explained rungs wherever a check could be written, added ratchets so that legacy violations could only shrink, and moved its release checks into a harness that the change under review cannot weaken. Figure 10 draws the ladder as it stands at the end of the paper: each rung with its teaching move and its Lean mechanism, the patterns that realise it, how both known uses bind it, where a broken rule is found, and underneath, the loop of trust, delegation and conformity that keeps it climbing.

We propose the ladder as a standard for repositories shared with coding agents, one answer to each of the five gaps in Section 9: a shared vocabulary of rungs; instruction files that state only what is not yet checked; checks chosen by risk and written to teach; requirements closed against scenarios; and violation seeding as a benchmark of the repository rather than of the agent that works in it.

Lean is why we think it works. Lean counts as value only

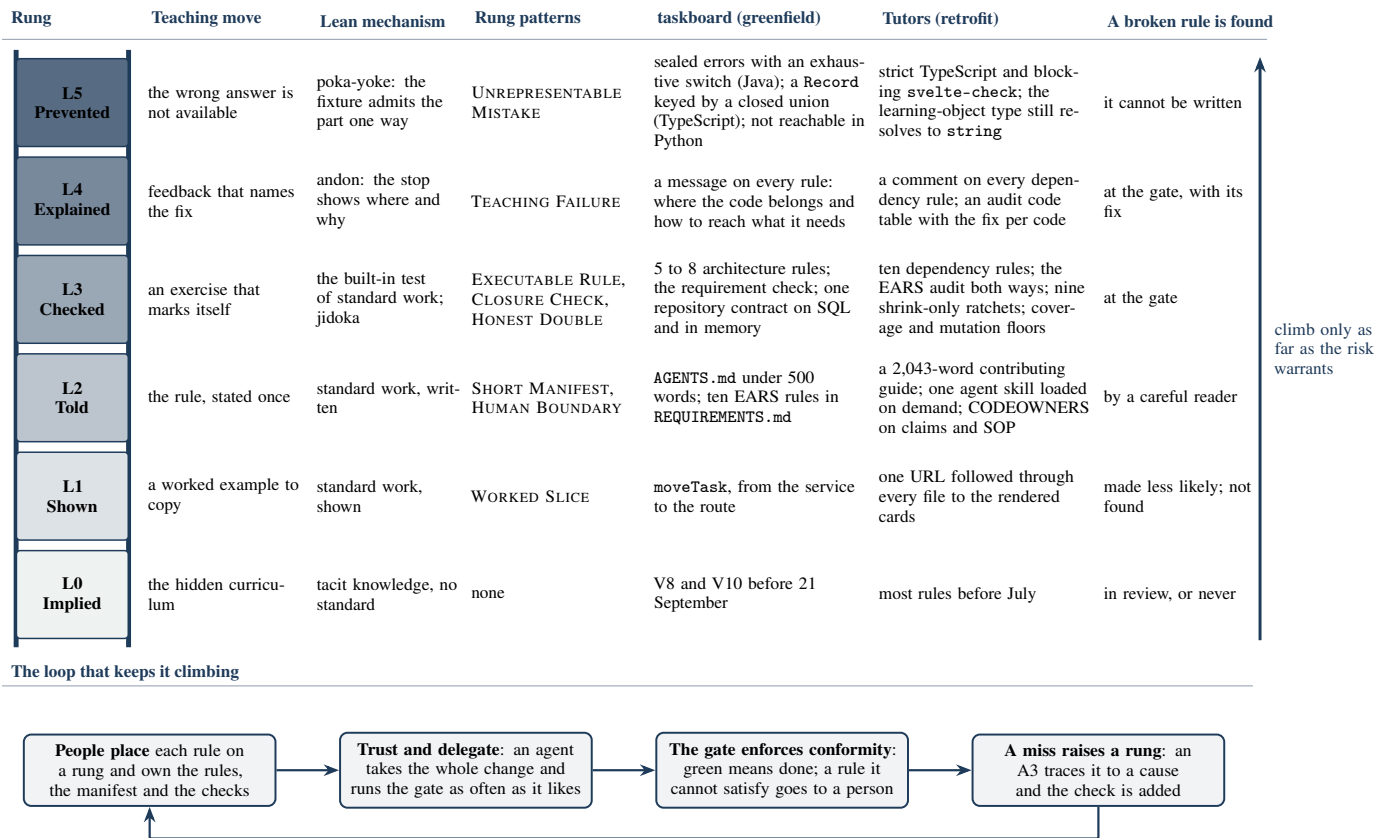


Figure 10 The Assertion Ladder, complete. Read a row for one rung across both known uses; read the bottom strip for how a team keeps climbing: people place each rule, an agent is trusted with the change, the gate enforces conformity, and a miss raises a rung.

what the customer, and here the domain, needs, and calls the rest waste. It makes the best current method standard work that everyone follows and the team improves. It asks where along the value stream a defect is found, and it answers each miss by asking why until the process can change. The ladder is standard work, with its tests built in, for a contributor who cannot remember it. Each rung a rule climbs moves detection further upstream. The two seeding runs are one turn of the improvement cycle, from a miss to its root cause to a countermeasure that was then measured again.

Four things come next. The pilot should be repeated against the current receivers, with more runs per cell and the feature request written in EARS, to test whether stated requirements improve placement. The harness-erosion use cases should be raised from L2 to L3 by protecting the rule files themselves. The release harness should be adopted by a second product, to find out how much of it is generic. And the reviewer's load, which we claim the ladder reduces by separating intent from code, should be measured rather than asserted.

Availability The receivers, the seeding script, the pilot's workspaces, audit and diffs, and this paper's sources are public (Griffin 2026a), as are Tutors and its release harness (Tutors contributors 2026a,b). Each receiver can be read on its own, starting from its AGENTS.md.

Acknowledgments

This paper is a collaboration between South East Technological University (SETU) and Red Hat in Waterford, Ireland.

References

Adzic, G. (2011). *Specification by example: How successful teams deliver the right software*. Manning.

AGENTS.md: A simple, open format for guiding coding agents. (n.d.). <https://agents.md>. (Accessed 2026)

Agrawal, L. A., Kanade, A., Goyal, N., Lahiri, S., & Rajamani, S. (2023). Monitor-guided decoding of code LMs with static analysis of repository context. In *Advances in neural information processing systems 36 (neurips)* (pp. 32270–32298).

Alexander, C., Ishikawa, S., & Silverstein, M. (1977). *A pattern language: Towns, buildings, construction*. Oxford University Press.

Ali, N. B., Petersen, K., & Schneider, K. (2016). FLOW-assisted value stream mapping in the early phases of large-scale software development. *Journal of Systems and Software*, 111, 213–227. doi: 10.1016/j.jss.2015.10.013

Bacchelli, A., & Bird, C. (2013). Expectations, outcomes, and challenges of modern code review. In *35th international conference on software engineering (icse)* (pp. 712–721). IEEE. doi: 10.1109/ICSE.2013.6606617

Barke, S., James, M. B., & Polikarpova, N. (2023). Grounded copilot: How programmers interact with code-generating

- models. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1). doi: 10.1145/3586030
- Baum, T., Schneider, K., & Bacchelli, A. (2019). Associating working memory capacity and code change ordering with code review performance. *Empirical Software Engineering*, 24(4), 1762–1798. doi: 10.1007/s10664-018-9676-8
- Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley.
- Becker, B. A., Denny, P., Pettit, R., Bouchard, D., Bouvier, D. J., Harrington, B., ... Prather, J. (2019). Compiler error messages considered unhelpful: The landscape of text-based programming error message research. In *Proceedings of the working group reports on innovation and technology in computer science education (iticse-wgr)* (pp. 177–210). ACM. doi: 10.1145/3344429.3372508
- Becker, J., Rush, N., Barnes, E., & Rein, D. (2025). *Measuring the impact of early-2025 AI on experienced open-source developer productivity*. arXiv:2507.09089. doi: 10.48550/arXiv.2507.09089
- Binamungu, L. P., Embury, S. M., & Konstantinou, N. (2018). Maintaining behaviour driven development specifications: Challenges and opportunities. In *Ieee 25th international conference on software analysis, evolution and reengineering (saner)* (pp. 175–184). IEEE. doi: 10.1109/SANER.2018.8330207
- Binamungu, L. P., & Maro, S. (2023). Behaviour driven development: A systematic mapping study. *Journal of Systems and Software*, 203, 111749. doi: 10.1016/j.jss.2023.111749
- Bjarnason, E., Runeson, P., Borg, M., Unterkalmsteiner, M., Engström, E., Regnell, B., ... Feldt, R. (2014). Challenges and practices in aligning requirements with verification and validation: A case study of six companies. *Empirical Software Engineering*, 19(6), 1809–1855. doi: 10.1007/s10664-013-9263-y
- Buschmann, F., Henney, K., & Schmidt, D. C. (2007). *Pattern-oriented software architecture, volume 5: On patterns and pattern languages*. Wiley.
- Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., & Stal, M. (1996). *Pattern-oriented software architecture, volume 1: A system of patterns*. Wiley.
- Chatlatanagulchai, W., Li, H., Kashiwa, Y., Reid, B., Thonglek, K., Leelaprute, P., ... Iida, H. (2025). *Agent READMEs: An empirical study of context files for agentic coding*. arXiv:2511.12884. doi: 10.48550/arXiv.2511.12884
- Chatlatanagulchai, W., Thonglek, K., Reid, B., Kashiwa, Y., Leelaprute, P., Rungsawang, A., ... Iida, H. (2025). On the use of agentic coding manifests: An empirical study of Claude Code. In *Proceedings of the 26th international conference on product-focused software process improvement (profes)*. (arXiv:2509.14744)
- Chen, M., et al. (2021). *Evaluating large language models trained on code*. arXiv:2107.03374.
- Chen, X., Lin, M., Schärli, N., & Zhou, D. (2024). Teaching large language models to self-debug. In *International conference on learning representations (iclr)*.
- Cockburn, A. (2005). *Hexagonal architecture*. <https://alistair.cockburn.us/hexagonal-architecture/>.
- Cunningham, W. (1992). The WyCash portfolio management system. In *Addendum to the proceedings on object-oriented programming systems, languages, and applications (oopsla '92)* (pp. 29–30). ACM. doi: 10.1145/157709.157715
- dependency-cruiser: Validate and visualise dependencies, with your rules. (n.d.). <https://github.com/sverweij/dependency-cruiser>. (Accessed September 2026)
- de Silva, L., & Balasubramaniam, D. (2012). Controlling software architecture erosion: A survey. *Journal of Systems and Software*, 85(1), 132–151. doi: 10.1016/j.jss.2011.07.036
- Dijkstra, E. W. (1968). The structure of the “THE”-multiprogramming system. *Communications of the ACM*, 11(5), 341–346.
- Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.
- Fakhoury, S., Naik, A., Sakkas, G., Chakraborty, S., & Lahiri, S. K. (2024). LLM-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Transactions on Software Engineering*, 50(9), 2254–2268. doi: 10.1109/TSE.2024.3428972
- Ford, N., Parsons, R., & Kua, P. (2017). *Building evolutionary architectures: Support constant change*. O'Reilly.
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The science of lean software and DevOps: Building and scaling high performing technology organizations*. IT Revolution.
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Gloaguen, T., Mündler, N., Müller, M., Raychev, V., & Vechev, M. (2026). Evaluating AGENTS.md: Are repository-level context files helpful for coding agents? In *Iclr 2026 workshop on memory for llm-based agentic systems (memagents)*. (arXiv:2602.11988)
- Gonçales, L., Farias, K., da Silva, B., & Fessler, J. (2019). Measuring the cognitive load of software developers: A systematic mapping study. In *Ieee/acm 27th international conference on program comprehension (icpc)* (pp. 42–52). IEEE. doi: 10.1109/ICPC.2019.00018
- Gotel, O. C. Z., & Finkelstein, A. C. W. (1994). An analysis of the requirements traceability problem. In *Proceedings of the ieee international conference on requirements engineering (icre)* (pp. 94–101). IEEE. doi: 10.1109/ICRE.1994.292398
- Griffin, L. (2026a). *AI reference projects: taskboard in Java, Python and TypeScript*. <https://github.com/lgriffin/AI-Reference-Projects>.
- Griffin, L. (2026b). *Build high value software systems*. Manning. <https://www.manning.com/books/build-high-value-software-systems>. (Manning Early Access Program; publication expected November 2026)
- Harding, W., & Kloster, M. (2024, January). *Coding on Copilot: 2023 data suggests downward pressure on code quality* (Tech. Rep.). GitClear. Retrieved from https://www.gitclear.com/coding_on_copilot_data_shows_ais_downward_pressure_on_code_quality

- 1 He, H., Miller, C., Agarwal, S., Kästner, C., & Vasilescu, B.
2 (2026). Speed at the cost of quality: How Cursor AI increases
3 short-term velocity and long-term complexity in open-source
4 projects. In *Proceedings of the 23rd international conference*
5 *on mining software repositories (msr)*. (arXiv:2511.04427)
- 6 Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press,
7 O., & Narasimhan, K. (2024). SWE-bench: Can language
8 models resolve real-world GitHub issues? In *International*
9 *conference on learning representations (iclr)*.
- 10 Kalyuga, S., Ayres, P., Chandler, P., & Sweller, J. (2003). The
11 expertise reversal effect. *Educational Psychologist*, 38(1),
12 23–31. doi: 10.1207/S15326985EP3801_4
- 13 Khurum, M., Petersen, K., & Gorschek, T. (2014). Extend-
14 ing value stream mapping through waste definition beyond
15 customer perspective. *Journal of Software: Evolution and*
16 *Process*, 26, 1074–1105. doi: 10.1002/smr.1647
- 17 Kirschner, P. A., Sweller, J., & Clark, R. E. (2006). Why
18 minimal guidance during instruction does not work: An anal-
19 ysis of the failure of constructivist, discovery, problem-based,
20 experiential, and inquiry-based teaching. *Educational Psy-*
21 *chologist*, 41(2), 75–86. doi: 10.1207/s15326985ep4102_1
- 22 Kruchten, P., Nord, R. L., & Ozkaya, I. (2012). Technical debt:
23 From metaphor to theory and practice. *IEEE Software*, 29(6),
24 18–21. doi: 10.1109/MS.2012.167
- 25 Lehman, M. M. (1980). Programs, life cycles, and laws of
26 software evolution. *Proceedings of the IEEE*, 68(9), 1060–
27 1076. doi: 10.1109/PROC.1980.11805
- 28 Li, H., Zhang, H., & Hassan, A. E. (2025). *The rise of*
29 *AI teammates in software engineering (SE) 3.0: How au-*
30 *tonomous coding agents are reshaping software engineering*.
31 arXiv:2507.15003. doi: 10.48550/arXiv.2507.15003
- 32 Li, R., Liang, P., Soliman, M., & Avgeriou, P. (2022). Under-
33 standing software architecture erosion: A systematic mapping
34 study. *Journal of Software: Evolution and Process*, 34(3),
35 e2423. doi: 10.1002/smr.2423
- 36 Liker, J. K. (2004). *The Toyota way: 14 management principles*
37 *from the world's greatest manufacturer*. McGraw-Hill.
- 38 Lulla, J. L., Mohsenimofidi, S., Galster, M., Zhang, J. M., Baltes,
39 S., & Treude, C. (2026). *On the impact of AGENTS.md files*
40 *on the efficiency of AI coding agents*. arXiv:2601.20404. doi:
41 10.48550/arXiv.2601.20404
- 42 Martin, R. C. (2017). *Clean architecture: A craftsman's guide*
43 *to software structure and design*. Prentice Hall.
- 44 Mathews, N. S., & Nagappan, M. (2024). Test-driven devel-
45 opment and LLM-based code generation. In *Proceedings*
46 *of the 39th ieee/acm international conference on automated*
47 *software engineering (ase)* (pp. 1583–1594). ACM. doi:
48 10.1145/3691620.3695527
- 49 Mavin, A., & Wilkinson, P. (2019). Ten years of EARS. *IEEE*
50 *Software*, 36(5), 10–14. doi: 10.1109/MS.2019.2921164
- 51 Mavin, A., Wilkinson, P., Harwood, A., & Novak, M. (2009).
52 Easy approach to requirements syntax (EARS). In *Pro-*
53 *ceedings of the 17th ieee international requirements en-*
54 *gineering conference (re'09)* (pp. 317–322). IEEE. doi:
55 10.1109/RE.2009.9
- 56 Meszaros, G. (2007). *xUnit test patterns: Refactoring test code*.
57 Addison-Wesley.
- Mohsenimofidi, S., Galster, M., Treude, C., & Baltes, S. (2026).
Context engineering for AI agents in open-source software. In
Proceedings of the 23rd international conference on mining
software repositories (msr). (arXiv:2510.21413)
- Mündler, N., He, J., Wang, H., Sen, K., Song, D., & Vechev, M.
(2025). Type-constrained code generation with language mod-
els. *Proceedings of the ACM on Programming Languages*,
9(PLDI), 601–626. doi: 10.1145/3729274
- Murphy, G. C., Notkin, D., & Sullivan, K. J. (2001). Software
reflexion models: Bridging the gap between design and im-
plementation. *IEEE Transactions on Software Engineering*,
27(4), 364–380. doi: 10.1109/32.917525
- Nguyen, N., & Nadi, S. (2022). An empirical evaluation of
GitHub Copilot's code suggestions. In *Proceedings of the*
19th international conference on mining software repositories
(msr). ACM. doi: 10.1145/3524842.3528470
- North, D. (2006). *Introducing BDD*. <https://dannorth.net/introducing-bdd/>. (First published as “Behavior Modification”
in Better Software magazine, March 2006)
- Ohno, T. (1988). *Toyota production system: Beyond large-scale*
production. Productivity Press.
- Parnas, D. L. (1972). On the criteria to be used in decomposing
systems into modules. *Communications of the ACM*, 15(12),
1053–1058.
- Passos, L., Terra, R., Valente, M. T., Diniz, R., & das Cha-
gas Mendonça, N. (2010). Static architecture-conformance
checking: An illustrative overview. *IEEE Software*, 27(5),
82–89. doi: 10.1109/MS.2009.117
- Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri,
R. (2022). Asleep at the keyboard? assessing the security
of GitHub Copilot's code contributions. In *Ieee symposium*
on security and privacy (s&p). doi: 10.1109/SP46214.2022
.9833571
- Peng, S., Kalliamvakou, E., Cihon, P., & Demirel, M. (2023).
The impact of AI on developer productivity: Evidence from
GitHub Copilot. arXiv:2302.06590. doi: 10.48550/arXiv
.2302.06590
- Perry, D. E., & Wolf, A. L. (1992). Foundations for the study of
software architecture. *ACM SIGSOFT Software Engineering*
Notes, 17(4), 40–52. doi: 10.1145/141874.141884
- Perry, N., Srivastava, M., Kumar, D., & Boneh, D. (2023). Do
users write more insecure code with AI assistants? In *Acm*
sigsaac conference on computer and communications security
(ccs). doi: 10.1145/3576915.3623157
- Poesia, G., Polozov, A., Le, V., Tiwari, A., Soares, G., Meek,
C., & Gulwani, S. (2022). Synchromesh: Reliable code gen-
eration from pre-trained language models. In *International*
conference on learning representations (iclr).
- Poppendieck, M., & Poppendieck, T. (2003). *Lean software*
development: An agile toolkit. Addison-Wesley.
- Poppendieck, M., & Poppendieck, T. (2006). *Implementing*
lean software development: From concept to cash. Addison-
Wesley.
- Reinertsen, D. G. (2009). *The principles of product development*
flow: Second generation lean product development. Redondo
Beach, CA: Celeritas Publishing.
- Renkl, A., & Atkinson, R. K. (2003). Structuring the transition

from example study to problem solving in cognitive skill acquisition: A cognitive load perspective. *Educational Psychologist*, 38(1), 15–22. doi: 10.1207/S15326985EP3801_3

Ries, E. (2011). *The lean startup: How today's entrepreneurs use continuous innovation to create radically successful businesses*. Crown Business.

Rother, M., & Shook, J. (1999). *Learning to see: Value-stream mapping to create value and eliminate muda*. Cambridge, MA: Lean Enterprise Institute.

Schäfer, M., Nadi, S., Eghbali, A., & Tip, F. (2024). An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 50(1), 85–105. doi: 10.1109/TSE.2023.3334955

Schmid, L., Hey, T., Armbruster, M., Corallo, S., Fuchß, D., Keim, J., ... Koziol, A. (2025). *Software architecture meets LLMs: A systematic literature review*. arXiv:2505.16697. doi: 10.48550/arXiv.2505.16697

Sedano, T., Ralph, P., & Péraire, C. (2017). Software development waste. In *Proceedings of the 39th international conference on software engineering (icse)* (pp. 130–140). IEEE. doi: 10.1109/ICSE.2017.20

Seddon, D. (n.d.). *Import linter*. <https://import-linter.readthedocs.io>. (Accessed September 2026)

Shingo, S. (1986). *Zero quality control: Source inspection and the poka-yoke system*. Productivity Press.

Shute, V. J. (2008). Focus on formative feedback. *Review of Educational Research*, 78(1), 153–189. doi: 10.3102/0034654307313795

Siegmund, J. (2016). Program comprehension: Past, present, and future. In *Ieee 23rd international conference on software analysis, evolution, and reengineering (saner)* (pp. 13–20). IEEE. doi: 10.1109/SANER.2016.35

Sobek II, D. K., & Smalley, A. (2008). *Understanding A3 thinking: A critical component of Toyota's PDCA management system*. Productivity Press.

Solis, C., & Wang, X. (2011). A study of the characteristics of behaviour driven development. In *37th euromicro conference on software engineering and advanced applications (seaa)* (pp. 383–387). IEEE. doi: 10.1109/SEAA.2011.76

Spear, S., & Bowen, H. K. (1999, September). Decoding the DNA of the Toyota production system. *Harvard Business Review*, 77(5).

Staats, B. R., Brunner, D. J., & Upton, D. M. (2011). Lean principles, learning, and knowledge work: Evidence from a software services provider. *Journal of Operations Management*, 29(5), 376–390. doi: 10.1016/j.jom.2010.11.005

Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2), 257–285. doi: 10.1207/s15516709cog1202_4

Sweller, J., & Cooper, G. A. (1985). The use of worked examples as a substitute for problem solving in learning algebra. *Cognition and Instruction*, 2(1), 59–89. doi: 10.1207/s1532690xci0201_3

Terra, R., & Valente, M. T. (2009). A dependency constraint language to manage object-oriented software architectures. *Software: Practice and Experience*, 39(12), 1073–1094. doi: 10.1002/spe.931

TNG Technology Consulting. (n.d.). *ArchUnit: Unit test your Java architecture*. <https://www.archunit.org>. (Accessed 2026)

Tutors contributors. (2026a). *Tutors mono repo: an open-source course reader and learning platform*. <https://github.com/tutors-sdk/tutors-mono-repo>. (Read at commit 865d02f. Accessed 2026-09-30)

Tutors contributors. (2026b). *Tutors release harness*. <https://github.com/tutors-sdk/tutors-release-harness>. (Version 1.15.0, read at commit c324525. Accessed 2026-09-30)

Vaithilingam, P., Zhang, T., & Glassman, E. L. (2022). Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Chi conference on human factors in computing systems, extended abstracts*. ACM. doi: 10.1145/3491101.3519665

Wang, X., Conboy, K., & Cawley, O. (2012). “Leagile” software development: An experience report analysis of the application of lean approaches in agile software development. *Journal of Systems and Software*, 85(6), 1287–1299. doi: 10.1016/j.jss.2012.01.061

Wang, X., Li, B., Song, Y., Xu, F. F., Tang, X., Zhuge, M., ... Neubig, G. (2025). OpenHands: An open platform for AI software developers as generalist agents. In *International conference on learning representations (iclr)*.

Womack, J. P., & Jones, D. T. (1996). *Lean thinking: Banish waste and create wealth in your corporation*. Simon & Schuster.

Wood, D., Bruner, J. S., & Ross, G. (1976). The role of tutoring in problem solving. *Journal of Child Psychology and Psychiatry*, 17(2), 89–100. doi: 10.1111/j.1469-7610.1976.tb00381.x

Wynne, M., & Hellesøy, A. (2012). *The cucumber book: Behaviour-driven development for testers and developers*. Pragmatic Bookshelf.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., & Press, O. (2024). SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in neural information processing systems (neurips)*.

Zhang, Y., Ruan, H., Fan, Z., & Roychoudhury, A. (2024). AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd acm sigsoft international symposium on software testing and analysis (issta)*. ACM. doi: 10.1145/3650212.3680384

A. Reading a receiver

Each receiver is self-contained. A reader interested in one language can start from its directory and read in this order.

1. AGENTS.md: the manifest. Verify command, placement table, conventions, requirement convention, feature recipe, and what needs a person first.
2. REQUIREMENTS.md: the ten rules in EARS, with ids and the pattern-to-layer table.

- 1 3. The worked slice: the domain entity, the service's move-
2 task method, the repository interface and its implementa-
3 tion, and the route or controller.
- 4 4. The scenarios: unit tests over the service with an in-
5 memory repository, and API tests through the real compo-
6 sition root. Each cites the requirements it covers.
- 7 5. The rules: the architecture tests and the requirement check.

8 The evaluation scripts are in `evaluation/`: `measure.py`
9 for `size`, `seed_violations.py` for seeding, and
10 `agent_experiment/` for the pilot. The first iteration's results
11 are kept alongside the current ones as `*-8c4a74a.json`.

Table 11 Where to find each part in each receiver.

Part	java/	python/	typescript/
Verify	<code>mvn verify</code>	<code>pytest</code>	<code>npm test</code>
Architecture rules	<code>ArchitectureTest.java</code>	<code>test_architecture.py</code> *	<code>architecture.test.ts</code> *
Requirement check	<code>RequirementsTest.java</code>	<code>test_requirements.py</code> *	<code>requirements.test.ts</code> *
Scenarios	<code>service/, api/, events/ tests</code>	<code>tests/unit, tests/api</code>	<code>tests/unit, tests/api</code>
Honest double	<code>TaskRepositoryContract.java</code>	<code>tests/integration/</code>	<code>tests/integration/</code>

* under `tests/architecture/`.

1 **About the authors**

2 **Leigh Griffin** leads the OCTO team at Red Hat in Waterford,
3 Ireland. You can contact the author at lgriffin@redhat.com.

4 **Paul Power** is a technical lead at Red Hat in Waterford, Ireland.
5 You can contact the author at ppower@redhat.com.

6 **Ray Carroll** is a Senior Principal Software Engineer at Red
7 Hat in Waterford, Ireland. You can contact the author at rcarroll@redhat.com.

9 **Colm Dunphy** leads the Higher Diploma in Science in Computer
10 Science at South East Technological University, Waterford, Ire-
11 land. You can contact the author at Colm.Dunphy@setu.ie.

12 **Darrin Taylor** is the Lean Enterprise Excellence Leader at South
13 East Technological University, Waterford, Ireland, and leads its
14 Master of Business Studies in Lean Enterprise Excellence. You
15 can contact the author at Darrin.Taylor@setu.ie.

16 **Eamonn de Leastar** leads the Higher Diploma in Science
17 in Computer Science at South East Technological Univer-
18 sity, Waterford, Ireland. You can contact the author at [Ea-
19 monn.Deleastar@setu.ie](mailto:Eamonn.Deleastar@setu.ie).